

$$\begin{aligned}
 \begin{bmatrix} h_0 & h_1 & h_2 & h_3 & 0 & 0 & 0 & 0 \\ x_0 & 0 & 0 & 0 & 0 & x_3 & x_2 & x_1 \end{bmatrix} & y_0 = h_0 x_0 \\
 \begin{bmatrix} h_0 & h_1 & h_2 & h_3 & 0 & 0 & 0 & 0 \\ x_1 & x_0 & 0 & 0 & 0 & 0 & x_3 & x_2 \end{bmatrix} & y_1 = h_0 x_1 + h_1 x_0 \\
 \begin{bmatrix} h_0 & h_1 & h_2 & h_3 & 0 & 0 & 0 & 0 \\ x_2 & x_1 & x_0 & 0 & 0 & 0 & 0 & x_3 \end{bmatrix} & y_2 = h_0 x_2 + h_1 x_1 + h_2 x_0 \\
 \begin{bmatrix} h_0 & h_1 & h_2 & h_3 & 0 & 0 & 0 & 0 \\ x_3 & x_2 & x_1 & x_0 & 0 & 0 & 0 & 0 \end{bmatrix} & y_3 = h_0 x_3 + h_1 x_2 + h_2 x_1 + h_3 x_0
 \end{aligned}$$

FIGURE 4.17

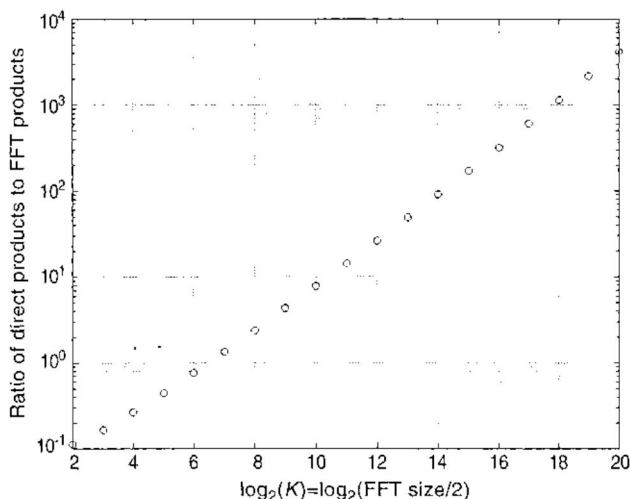
Illustration of the periodic convolution of vectors h and x as in Figure 4.15, but with zero extension, so K is increased from four to eight.

Before going further with this, we may ask whether the FFT method will really save time, because the vectors are all twice as large. The answer depends on the size and value of K , as suggested in Chapter 3, Figure 3.5. Using the number of complex products as a measure of computing time, we can see in (4.65) that there are K products per sum and K sums; therefore, K^2 products in the convolution. Assuming the best case where K (and hence, $2K$) is a power of two, (3.17) with $N = 2K$ gives $K \log_2(2K)$ complex products per FFT. A complex product in Cartesian form requires four products, so we use $4K \log_2(2K)$ as the number of products per FFT. In (4.68), there are two FFTs and one inverse FFT, for a total of $12K \log_2(2K)$ products. Therefore,

$$\frac{\# \text{ products using (4.65)}}{\# \text{ products using (4.68)}} = \frac{K^2}{12K \log_2(2K)} = \frac{K}{12(1 + \log_2 K)} \quad (4.69)$$

This ratio is plotted in Figure 4.18 for cases where K is a power of 2. When the ratio exceeds one, the FFT method becomes profitable in terms of computing time. Thus, the FFT method is worth considering for $K \geq 128$, and definitely preferable for very large values of K .

Seeing that the FFT method can be useful for large values of K , we look in more detail first at the *nonrecursive* case, where (4.65) describes the linear system as a single convolution with $h = b$, the nonrecursive weight vector. In practical linear operations where K is large, which we assume here, the number of weights is usually less than K . Suppose this is true. To see the effect on periodic convolution, suppose $h_2 = h_3 = 0$ in Figure 4.17. Then, we would not need as many zeros to avoid circular convolution. In fact, we only need to extend x with N zeros, where N is the number of nonrecursive weights. So, assuming we still require the FFT size to be a power of two, we can

**FIGURE 4.18**

Ratio of products required for convolution using the direct method (4.65) to products required using the FFT method (4.68), when the vector length, K , is a power of two. The FFT method becomes profitable as K increases from 64 to 128.

modify (4.68) as follows:

Nonrecursive FFT Filtering; $K \gg N$

$N = \text{length}(b)$

$K = \text{length}(x)$

$L = \text{next power of two} \geq K + N$

$y = \text{IFFT}[\text{FFT}(b, L) \cdot \text{FFT}(x, L)]$

$y = \text{real}(y(1:K))$

(4.70)

In (4.70), the FFTs are smaller than in (4.68), but the convolution in (4.65) now has N products per sum instead of K products per sum. So Figure 4.18, while no longer valid for this case, is still an approximation. The last line of (4.70) is written to adjust y to its correct length and to remove small imaginary components that are the result of rounding errors.

In the *recursive* case, direct processing as in (4.10) consists of two convolutions. Assume $M = N$ so there are $2N$ products per sum, or a total of $2NK$ products in the direct form. In the FFT version, we must first find h as the inverse transform of $B(z)/A(z)$. Division of FFTs is not a good idea, because even assuming the system is stable, $A(z)$ could have zeros near the unit circle at points where the FFTs are computed, resulting in an inaccurate computation of h .

The preferred method for finding h with a recursive system is *polynomial division*, in which the division of $B(z)$ by $A(z)$ is simply carried out by long division to the desired length of h . The process is easy to encode in languages like MATLAB that operate on vectors. A function called *imp_resp* is included to illustrate the process. Once $h(1:K)$ is found, it may be used as the nonrecursive weight vector in place of b in (4.70) to implement recursive filtering using FFTs.

If MATLAB is being used to do the processing, the MATLAB *conv*, *deconv*, and *filter* functions, which use compiled algorithms, should be used, because, with long signal vectors, they are at least an order of magnitude faster than *imp_resp* and the expressions in (4.70). (See also the *filters* function described at the end of Section 6.7 in Chapter 6.)

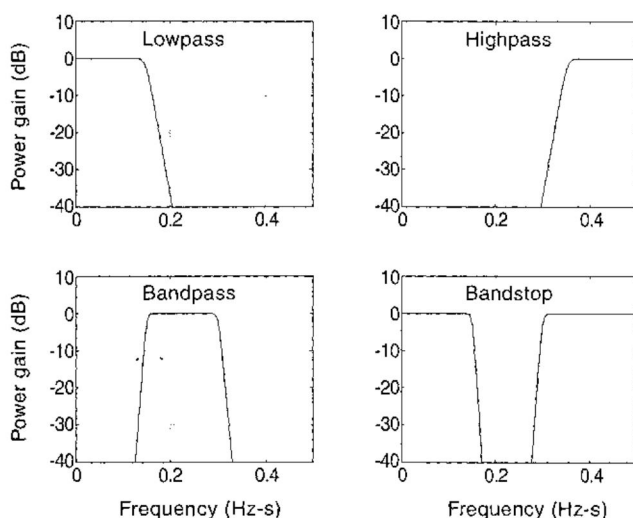
On the other hand, if a processor is using a primitive language, and particularly if a 2^k -element FFT chip is available, then *block FFT filtering* using the algorithm in (4.70) should allow processing at the highest possible sample rates. In block FFT filtering, as the name suggests, the input signal is processed in successive blocks (vectors) of length 2^k in order to use the FFT chip most efficiently.

The exercises include applications of FFT filtering with recursive and nonrecursive filters. The MATLAB language provides an excellent set of tools to test such applications, as we saw in (4.70), which is almost a set of MATLAB expressions. The main purpose of this section has been to show that, essentially as the result of the relationship in (4.8) (that is, convolution in the time domain implies multiplication in the frequency domain), the efficiency of the FFT algorithm may be applied to digital filtering.

4.13 Discrete Linear Systems and Digital Filters

The names *digital filter* and *discrete linear system* refer to signal processing systems like those we have been discussing in this chapter. But when we use *filter*, we usually are thinking of a process analogous to filtering in chemistry, where some of the ingredients of a mixture are removed by a filter to produce a mixture of greater purity. In signal processing, the mixture is a signal mixed with noise. The filter is a discrete processor, and the desired output is the signal with less noise.

In the next two chapters, we discuss filters in which the discrete processor is a linear nonrecursive (FIR) or recursive (IIR) system of the type discussed in this chapter. When a linear processor of this type is used as a filter, its effect is most evident in the frequency domain—particularly in the amplitude response characteristic. Certain frequency components of the input signal are allowed to pass through the filter, while other components are attenuated by the filter. *Power gain* characteristics of the four types of filters used most

**FIGURE 4.19**

Power gain characteristics of four common types of digital filters. The filter weights were determined using algorithms described in Chapter 6.

commonly in DSP are illustrated in Figure 4.19. Power gain is the square of amplitude gain and is usually measured in *decibels* (dB), that is,

$$\begin{aligned} \text{Power gain in dB} &= 10 \log_{10} \left| H\left(j\frac{2\pi\nu}{T}\right) \right|^2 \\ &= 20 \log_{10} \left| H\left(j\frac{2\pi\nu}{T}\right) \right| \end{aligned} \quad (4.71)$$

where $|H(j\omega)|$ is the amplitude gain in (4.13), and $\nu = \omega T/2\pi$ Hz-s. Power gain plots are commonly used in the literature to illustrate filter gain properties. In each case, the name of the filter in Figure 4.19 describes the gain property (the processing effect of the filter).

The filter weights used for the plots in Figure 4.19 were computed using methods described in Chapter 6; thus, the plots are for typical recursive digital filters. From (4.71), we can see that a power gain equal to 0 dB translates to an amplitude gain equal to one; a power gain equal to -40 dB translates to an amplitude gain equal to 0.01.

There are also digital filters that are not linear processors. Nonlinear filters such as neural networks and time-varying or adaptive filters are examples. The discussion of these is left to Chapter 9 in the case of adaptive filters and to texts more specialized in specific areas of DSP.

4.14 Exercises

General Instructions: (1) Whenever you make a plot in any of the exercises, be sure to label both axes so we can tell exactly what the units mean. (2) Make continuous plots unless otherwise specified. "Discrete plot" means to plot discrete symbols, unconnected unless otherwise specified. (3) In plots, and especially in subplots, use the *axis* function for best use of the plotting area.

1. Find a nonrecursive expression of the linear system described by $y_k = x_k + x_{k-1} + y_{k-1}$. Assume zero initial conditions at $k = 0$. Then present a clear inductive proof of your result.
2. Decide whether the first four properties in Table 4.1 exist for the following systems:
 - a. $y_k = b \sin x_{k-1}$
 - b. $y_k = bx_{k-1} - ay_{k-2}$
 - c. $y_k = b_1x_{k-1} + b_2x_k + b_3x_{k+1}$
 - d. $y_k = x_k + bx_{k-1}y_{k-1}$
3. Suppose the sample vector x , which begins at $k = 0$, is $x = [1 \ 1 \ -1 \ 0 \ 0 \ \dots]$. Show the startup sequences for linear systems (c) and (d) in Exercise 2.
4. Write the transfer function, $H(z)$, for linear systems (b) and (c) in Exercise 2. For (b), express the transfer function as a ratio of polynomials in z .
5. The transfer function of a digital filter is given by

$$H(z) = \frac{0.0675 + 0.1350z^{-1} + 0.0675z^{-2}}{1 - 1.1430z^{-1} + 0.4128z^{-2}}$$

- a. Using (4.12), show that the filter gain is one at $\omega T = 0$ and zero at $\omega T = \pi$.
- b. Plot the amplitude gain versus frequency in rad, and verify that your answer to (a) is correct. Use zero extension with an FFT size of 1000 in a manner similar to (4.14).
6. For the filter in Exercise 5, plot phase shift in degrees versus frequency in Hz-s again in a manner similar to (4.14), but this time, make a plot of connected discrete points using an FFT size of 100. Note that the phase shift at 0.5 Hz-s is indeterminate, because the gain at this frequency is zero.
7. Make a connected discrete plot of the complete impulse response of the filter in Exercise 5, from its beginning until it becomes negligible. Normalize the plot so its maximum value is 1.0. Also, print

the normalized impulse response and identify the point where its magnitude stays below 1%.

8. The weights of a digital filter in the form of (4.11) are as follows:

```
b=[0.0675    0.0000   -0.1349    0.0000    0.0675];
```

```
a=[1.0000   -1.9425    2.1192   -1.2167    0.4128].
```

Plot the continuous amplitude response versus frequency in terms of KHz, assuming a sampling frequency of 40 KHz.

9. Make a continuous plot of the phase shift of the filter in Exercise 8, in terms of degrees versus frequency in KHz, assuming the filter is working with a time step of 0.025 ms.
10. Make a connected discrete plot of the complete impulse response of the filter in Exercise 8, from its beginning until it becomes negligible. Normalize the plot so its maximum magnitude is 1.0. Also, print the normalized impulse response and identify the point where its magnitude stays below 1%.
11. Plot the poles and zeros of the digital filter in Exercise 8 using the *pz_plot* function. Explain the following:
 - a. How did the pole and zero locations indicate a bandpass gain characteristic?
 - b. What is the approximate frequency range of the passband?
 - c. Why is the phase shift zero at $\omega = 0$ and then jumps to 180° when ω is just above zero?
12. Write a MATLAB function $[H, nu] = my_gain(b, a)$ to compute the complex gain of any linear system, recursive or nonrecursive, in terms of the b and a vectors in (4.12). Have the function return N values of H and ν , evenly spaced from zero through half the sampling rate, including the endpoints, with N set internally to 1024. Verify that the function works by duplicating the left-hand plot in Figure 4.2. If the function works, you may use it as desired in subsequent applications.
13. Write a MATLAB function $[dB, nu] = my_power_gain(b, a)$ to compute power gain in dB versus frequency in Hz-s for any linear system. Have the function return N values of dB and ν , with N set internally to 1024. Write the function so that it limits the range of dB to 200. Test the function on the following vectors:

```
b=[0.0675    0.1350    0.0675]
```

```
a=[1.0000   -1.1430    0.4128]
```

Plot dB versus ν , and use *axis*([0 0.5 -80 0]) to set the ν and dB axis limits. If your plot shows 0 dB at $\nu = 0$ and -25 dB at $\nu = 0.3$, your function is probably correct, and you may use it to make power gain plots in subsequent chapters.

14. A linear transfer function is given as $H(z) = (z^2 + 1)/(z^2 - 1.93z + 1.21)$. Make two subplots. On the left, plot the poles and zeros of $H(z)$. On the right, plot the first 60 samples of the impulse response. Discuss the stability of $H(z)$ in terms of each plot.
15. Using lines B, 4, and 5 in Table 4.2, find the z -transform, $X(z)$, of $x_k = 2\sin(\omega k + \pi/6)$.
16. Two linear systems, $H_1(z) = 1 + 0.4z^{-1}$ and $H_2(z) = -bz^{-1}$, are connected together to form $H(z)$ in the feedback configuration of Figure 4.7.
 - a. On the complex z -plane, make a pole-zero plot showing the trajectory of the poles as b varies from 0 to 1.7.
 - b. For what range of b is $H(z)$ stable?
 - c. For what range of b are the poles of $H(z)$ real?
17. The direct form of a particular linear system is given by the following:

$$H(z) = \frac{2 + 1.1z^{-1} - 0.5z^{-2}}{1 + 0.1z^{-1} - 0.3z^{-2}}$$

- a. Express $H(z)$ in cascade form with two sections.
 - b. Express $H(z)$ in parallel form with two sections.
18. A linear system has the transfer function $H(z) = (0.5 + 0.6z^{-2})/(1 - 0.4z^{-2})$.
 - a. Is the system stable?
 - b. Write the difference equation for the system, and make a diagram similar to Figure 4.9.
 - c. Make a diagram similar to Figure 4.10, including the difference equations.
 - d. Make a diagram similar to Figure 4.11, including the difference equations.
19. Define the necessary arrays and write the state-space equations for the linear system in Exercise 18.
20. Do the following:
 - a. Find the weights of the lattice form of $H(z) = (1 + z^{-1})(1 - 0.8z^{-1})$.
 - b. Make diagrams of the direct and lattice forms.
 - c. Check for equivalence by computing the output vectors of each form when the input signal vector is $[0 \ 1 \ 3 \ -1 \ 0 \ 0 \ 0]$.
21. Complete the following:
 - a. Convert

$$H(z) = \frac{1 + 1.1z^{-1} - 0.5z^{-2}}{6(1 + 0.1z^{-1} - 0.8z^{-2})}$$

- to a lattice filter, and draw the lattice diagram, showing all weights.
- Make connected discrete plots of the direct and lattice impulse responses in two subplots, one above the other, to show the equivalence of the two forms.
- Complete the following:
 - Convert $H(z) = 1 + z^{-3} - z^{-4}$ to a nonrecursive lattice filter, and draw the direct and lattice diagrams, showing all weights.
 - Compute and print the direct and lattice impulse responses to show the equivalence of the two forms.
 - Plot the power gain in dB versus Hz-s of $H(z)$ in Exercise 21, which is the transfer function of a certain cable. Describe the effect of the cable when it is used to transmit signals.
 - The impulse response of a filter is given as follows:

$$h_k = \frac{\sin(1.25(k-25))}{3(k-25)}; \quad k = 0, 1, \dots, 50$$

- In the upper plot of three subplots, plot the following signal:

$$x_k = 10 \sin(0.5\pi k) + \cos(0.1\pi k); \quad k = 0, 1, \dots, 499$$
 - Use the FFT filtering algorithm in (4.70) to produce y , the result of filtering x with the filter just described. Plot y below x in the center plot.
 - Plot the amplitude gain of the filter in the lower plot, and explain what the filter did to x .
- Do Exercise 24 again, but this time use the following IIR filter:

$$H(z) = \frac{(0.2533z^2 + 0.5066z + 0.2533)(0.1839z^2 + 0.3678z + 0.1839)}{(z^2 - 0.4531z + 0.4663)(z^2 - 0.3290z + 0.0646)}$$

In the FFT product, use an impulse response having the same length as x .

References

- Oppenheim, A.V. and Schaffer, R.W., *Discrete-Time Signal Processing*, Prentice Hall, Englewood Cliffs, NJ, 1989, chaps. 4, 5.
- Orfanidis, S.J., *Introduction to Digital Signal Processing*, Prentice Hall, Upper Saddle River, NJ, 1996, chap. 5.

3. Stearns, S.D. and Hush, D.R., *Digital Signal Analysis*, Prentice Hall, Englewood Cliffs, NJ, 1989, chap. 7.
4. Churchill, R.V., Brown, J.W., and Verhey, R.F., *Complex Variables and Applications*, McGraw-Hill, New York, 1976.
5. Gray, A.H. and Markel, J.D., Digital lattice filter synthesis, *IEEE Trans. Audio Electroacoust.*, AU-21, 491, Dec. 1973.
6. Itakura, F. and Saito, S., Digital filtering techniques for speech analysis and synthesis, *Proc. 7th Int. Cong. Acoust.*, Budapest, 261, 1971.
7. Makhoul, J., Stable and efficient lattice methods for linear prediction, *IEEE Trans. Acoust. Speech Signal Process.*, ASSP-25, 423, Oct. 1977.
8. Mitra, S.K., Kamat, P.S., and Huey, D.C., Cascaded lattice realization of digital filters, *IEEE Trans. Circuit Theory Appl.*, 5, 3, 1977.
9. Jury, E.I., A note on the reciprocal zeros of a real polynomial with respect to the unit circle, *IEEE Trans. Commun. Tech.*, CT-11, 292, June 1964.
10. Oppenheim, A.V. and Willsky, A.S., *Signals and Systems*, Prentice Hall, Englewood Cliffs, NJ, 1983.
11. Schwarz, R.J. and Friedland, B., *Linear Systems*, McGraw-Hill, New York, 1965.
12. Moon, T.K. and Stirling, W.C., *Mathematical Methods and Algorithms for Signal Processing*, Prentice Hall, Upper Saddle River, NJ, 2000.

FIR Filter Design

5.1 Introduction

In Chapter 4, we saw that the transfer function of a linear nonrecursive or FIR filter is the transform of the filter's impulse response vector, the latter being equal to the filter's weight vector. In this chapter, we discuss the simplest FIR design techniques, which are based simply on starting with a desired transfer function in the form of a discrete Fourier transform (DFT) and then obtaining the filter weights via the inverse DFT. Because many different applications take the form of one of the four filters in Figure 4.19—lowpass, highpass, bandpass, and bandstop—we focus in this chapter on these four basic types of filters.

Compared with IIR filters, FIR filters have advantages and disadvantages. The major disadvantage is the result of the FIR transfer function being a polynomial in z and z^{-1} rather than a ratio of polynomials, and thus having zeros anywhere but poles only at the origin. Without poles away from the origin, more weights are generally required to provide a transition with given steepness from passband to stopband.

One major advantage of FIR filters comes in a property called *linear phase*, which we defined in Chapter 3 in terms of a shift in the time domain. We showed that when a signal x is delayed n time steps, $n\omega T$ radians are added to the phase of x , that is, added to the angle of the transform of x . This means that if x is expressed as a Fourier series, each component of x is adjusted in phase to produce a delay of exactly n time steps. So then, if the transfer function of a linear digital filter is given by

$$H(e^{j\omega T}) = |H(e^{j\omega T})|e^{-jn\omega T} \quad (5.1)$$

then, in a like manner, the effect of the filter will be to shift each component of the input signal by n time steps while modifying its amplitude. Thus, a linear-phase digital filter always has a transfer function in the form of (5.1). As we shall see, it is easy to impose this requirement on the design of FIR filters. We will also see in the next chapter that linear phase shift is not possible in stable IIR filters that operate in real time; therefore, linear phase is a major consideration in choosing the FIR construction.

FIR filters have other advantages as well. Having only zeros, they are not unstable as long as the weights are finite. Also, the finite impulse response property is often an advantage, because we can always specify the exact *startup time*, that is, the duration of the filter's response to a transient signal.

5.2 An Ideal Lowpass Filter

To derive the filter weights in the manner just described, as the inverse transform of the filter transfer function, we begin with the ideal transfer function shown in Figure 5.1, which is ideal in two ways: first, the gain is exactly one at frequencies $|\nu| \leq \nu_c$ Hz-s and exactly zero for $|\nu| > \nu_c$ Hz-s, and second, the transfer function is real, implying zero phase shift.

The units in Figure 5.1 require some discussion. The frequency units are Hz-s. (It is helpful here to recall the frequency conversions in Table 3.1, especially $\omega = 2\pi\nu T$.) The filter gain, which is real, is expressed (for convenience) as a continuous Fourier transform. The cutoff frequency, which is illustrated here at $\nu_c = 0.1$ Hz-s, is below half the sampling rate, that is, $\nu_c < 0.5$, and as long as this is true, the sampling theorem holds, and (3.42) gives a simple relationship between the Fourier transform and the DFT of the impulse samples, that is,

$$\bar{H}_{ideal}(j\omega) = \frac{1}{T} H_{ideal}(j\omega)$$

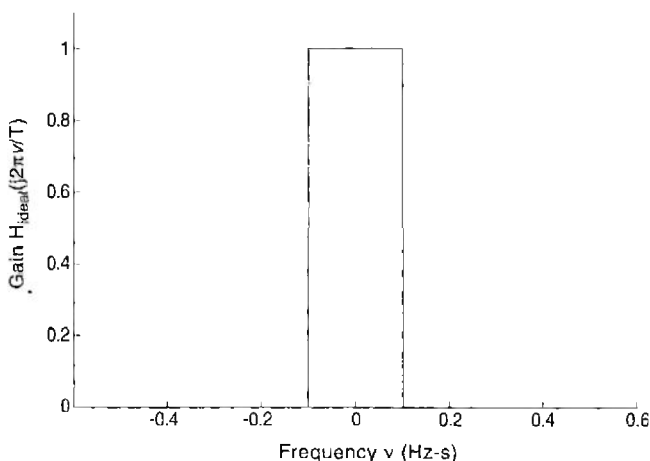
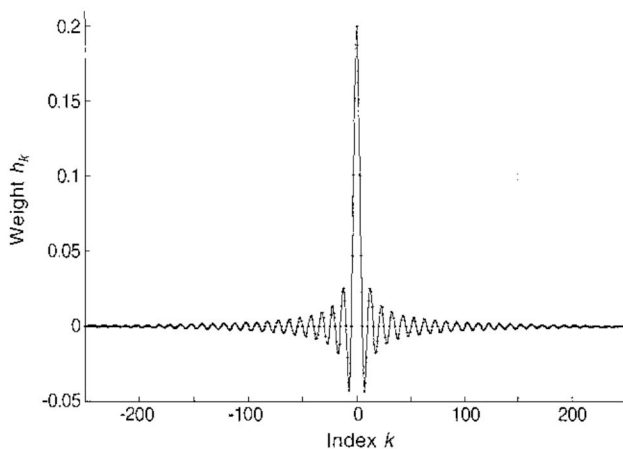


FIGURE 5.1

Gain of an ideal lowpass filter. The filter passes all signal components below $\nu_c = 0.1$ Hz-s without alteration, and stops all components above ν_c Hz-s.

**FIGURE 5.2**

Weight vector of the digital version of the ideal lowpass filter in Figure 5.1. Weights extend to $\pm\infty$.

Accordingly, because we wish $\bar{H}_{ideal}$ rather than H_{ideal} to be one at frequencies below ω_c , the discrete samples of the impulse response of this ideal filter must be T times the inverse Fourier transform of the transfer function; that is, according to (3.34),

$$\begin{aligned}
 h_{ideal}(kT) &= \frac{T}{2\pi} \int_{-\infty}^{\infty} H(j\omega) e^{j\omega kT} d\omega = \frac{T}{2\pi} \int_{-2\pi\nu_c/T}^{2\pi\nu_c/T} (1) e^{j\omega kT} d\omega \\
 &= \begin{cases} 2\nu_c; & k = 0 \\ \frac{\sin(2\pi\nu_c k)}{\pi k}; & 1 \leq |k| < \infty \end{cases} \quad (5.2)
 \end{aligned}$$

The central part of the ideal weight vector corresponding with Figure 5.1, $b_{ideal}(k) = h_{ideal}(kT)$, is plotted in Figure 5.2. Note that the maximum weight is $b_0 = 2\nu_c$, and that the weights extend infinitely in both directions.

5.3 The Realizable Version

The simplest way to make a realizable FIR filter out of the infinite weight vector in (5.2) is simply to truncate the vector at, say, $\pm L$, so there are $N = 2L + 1$ weights in the realizable version. We can see from (3.42) that the phase of the truncated vector is still zero as long as the weights remain symmetric about

zero, because each term in the vector's DFT sum is added to its complex conjugate, resulting in a real transfer function. Furthermore, if the entire weight vector is shifted L steps so that h_{-L} becomes h_0 , the result is a *causal* digital filter with *linear phase shift*, in accordance with Table 3.3, Property 8. In other words,

Weights of any causal FIR filter with linear phase shift: $b = [b_0 \ b_1 \ \dots \ b_{2L}]$ with $b_{L-n} = b_{L+n}; \quad n = 1, 2, \dots, L$ Total number of weights: $N = 2L + 1$.	(5.3)
---	-------

Using (5.2), the impulse response (or weights) of our realizable lowpass causal, linear-phase filter may be given by

$$h_k = \begin{cases} 2v_c; & k = L \\ \frac{\sin(2\pi v_c(k-L))}{\pi(k-L)}; & k = 0, 1, \dots, L-1, L+1, \dots, 2L \end{cases} \quad (5.4)$$

For weights computed in this manner, with $v_c = 0.1$ Hz-s, the power gain characteristic is plotted in Figure 5.3 for $L = 4, 8$, and 128 . The plots illustrate the effects of truncating the weight vector. First, the gain curve becomes less rectangular as L is decreased. Second, there is significant *passband ripple* and

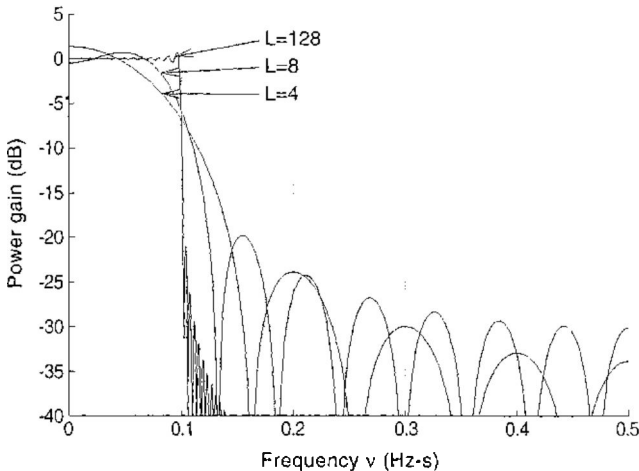


FIGURE 5.3

Power gain of three lowpass FIR filters, each with cutoff at 0.1 Hz-s. The number of weights in each case is $N = 2L + 1$.

stopband ripple, both of which are undesirable. Third, the power gain at cutoff is ideally -6 dB, corresponding with ideal amplitude gain 0.5, that is, halfway from one to zero in Figure 5.1. This also may be altered by truncation, that is, by the undesirable ripple in the gain characteristic.

The first effect, the nonrectangular gain curve, can obviously be improved by increasing L , but a larger L means more hardware and more computation per sample.

There is also another less obvious way to improve the shape of the gain curve, if the sampling rate can be adjusted. Suppose you are examining a digital processing system and notice that the analog-digital converter is set to convert at 10^8 samples per second; the sampling frequency is $f_s = 100$ MHz. Suppose you also know there is never any content above 5 MHz in the analog input signal. Then, obviously, f_s is about 10 times higher than required, and the filter gain, which is the DFT of the weights, is spread over 10 times the necessary range. Thus, the gain may be made more rectangular in cases like this by reducing the sampling frequency to a more reasonable value, say 10 MHz, which still avoids aliasing. An example is given in Figure 5.4. The upper plot is the power gain with $f_s = 100$ MHz, and the lower plot is the power gain with $f_s = 10$ MHz.

Even if the analog-to-digital conversion rate cannot be changed, f_s can be reduced simply by *decimation*, or *down-sampling*, provided that aliasing is not introduced. In the example just discussed, the process is illustrated in

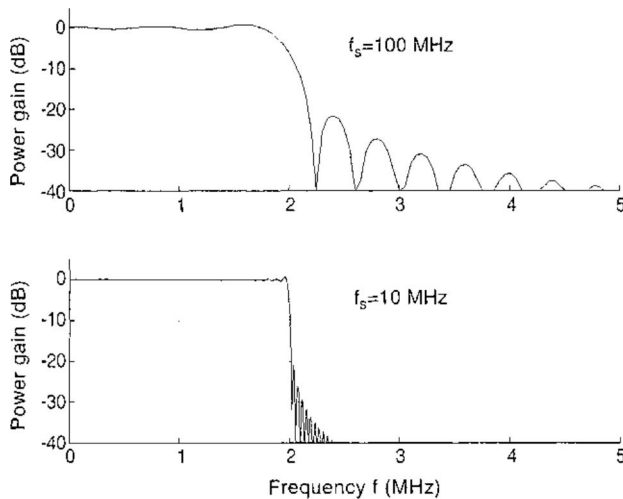


FIGURE 5.4

Power gain of two lowpass FIR filters, each with $N = 257$ weights and cutoff frequency $f_c = 2$ MHz. The upper filter is designed for sampling frequency $f_s = 100$ MHz and the lower for $f_s = 10$ MHz.

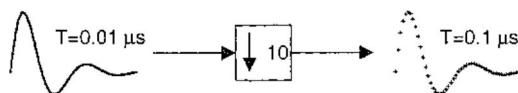


FIGURE 5.5

Down sampling. The sampling rate is reduced from 100 MHz to 10 MHz.

Figure 5.5. Down-sampling is represented by a box with a down-arrow and a number indicating the output/input time-step ratio. Aliasing is not introduced in this case, and the result is a waveform that we can filter successfully with fewer weights, as suggested in Figure 5.4.

When down-sampling with time-step ratio K is accomplished by decimation, $K - 1$ out of K samples of the input are ignored in each output time step. An obvious alternative to decimation is *averaging*, in which each output sample is the average of K input samples. We will see later that averaging is preferable to decimation, because it usually reduces the quantizing error of the analog-to-digital converter.

The second adverse effect caused by truncating the weight vector (the passband and stopband ripple evident in Figure 5.3) cannot be improved by the methods just discussed but may be improved and almost eliminated with the use of *window functions*, which is the subject of the next section.

5.4 Improving the Lowpass FIR Filter with Window Functions

The name *window function* comes from the idea of taking an infinite impulse response function such as $[h_k]$ in Figure 5.2 and superimposing a rectangular frame, like a window frame, through which you can only see impulse response values from index $k = -L$ to L . A *rectangular window* with length $2L + 1$ has the effect of multiplying $h(-L:L)$ by one and the rest of the h 's by zero. There are also *nonrectangular windows*, and these have the effect of scaling the weights within the window frame.

We have already seen how the rectangular window is used to obtain a realizable lowpass FIR filter with noticeable passband and stopband ripple. Our goal in this section is to apply nonrectangular windows to the ideal impulse response function and obtain a smoother gain characteristic with less ripple.

The theory we need for this application is similar to the convolution result in (4.67), that is, the DFT of the convolution of periodic functions in the time domain is the product of the two individual DFTs. Suppose we now take the DFT of the product of the truncated ideal impulse response,

$[h_k; k = 0, 1, \dots, N-1]$, and any window function, $[w_k; k = 0, 1, \dots, N-1]$:

$$\begin{aligned}
 \text{DFT}\{h_k w_k\} &= \sum_{k=0}^{N-1} h_k w_k e^{-j2\pi mk/N} \\
 &= \frac{1}{N^2} \sum_{k=0}^{N-1} \sum_{i=0}^{N-1} H_i e^{j2\pi ik/N} \sum_{n=0}^{N-1} W_n e^{j2\pi nk/N} e^{-j2\pi mk/N} \\
 &= \frac{1}{N^2} \sum_{i=0}^{N-1} H_i \sum_{n=0}^{N-1} W_n \sum_{k=0}^{N-1} e^{-j2\pi(i+n-m)k/N} \\
 &= \frac{1}{N} \sum_{i=0}^{N-1} H_i W_{m-i}; \quad m = 0, 1, \dots, N-1
 \end{aligned} \tag{5.5}$$

On the second line, inverse DFTs are substituted for the time functions. On the third line, the final sum is N when $n = m - i$ and zero otherwise. Thus, the DFT of a time-domain product is the convolution of individual DFTs, and taken together, (4.67) and (5.5) illustrate the duality of convolutions and transforms in the time and frequency domains. If x and y are vectors of length N ,

Convolution in the time and frequency domains:

$$\text{DFT}^{-1}\{X_m Y_m\} = \sum_{n=0}^{N-1} x_n y_{k-n+vN}; \quad k = 0, 1, \dots, N-1; \quad v = 0, 1, \dots \tag{5.6}$$

$$\text{DFT}\{x_k y_k\} = \frac{1}{N} \sum_{n=0}^{N-1} X_n Y_{m-n}; \quad m = 0, 1, \dots, N-1 \tag{5.7}$$

Notice that both convolutions involve periodic functions. In the convolution in (5.7), the DFT is always periodic as stated in Table 3.3, line 1.

Thus, we see in (5.5) that the gain of the FIR filter is the convolution of the DFT of the truncated ideal impulse response, $h(0:N-1)$, with the DFT of the window vector, $w(0:N-1)$. The DFT of the rectangular window has high side lobes, hence, the ripple in the filter gain function.

We illustrate this effect on the FIR lowpass power gain function using the four window functions illustrated with $N = 31$ in Figures 5.6 and 5.7. The relatively low value of N is used so the structures of the power spectra may be observed easily. The windows are in Figure 5.6, and their power spectra are in Figure 5.7. We use *boxcar*, the MATLAB term which is descriptive and easy to remember, for *rectangular* in Figure 5.6. In Figure 5.7, each of the corresponding window functions was scaled so the power gain was 0 dB at $v = 0$.

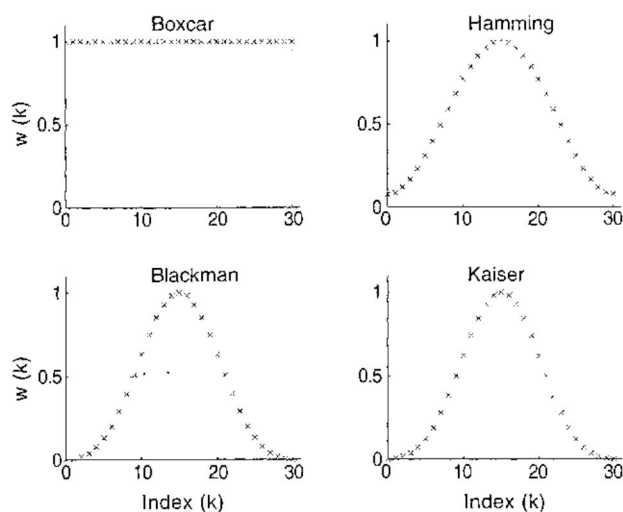


FIGURE 5.6

Rectangular (boxcar), Hamming, Blackman, and Kaiser window functions; $N = 31$.

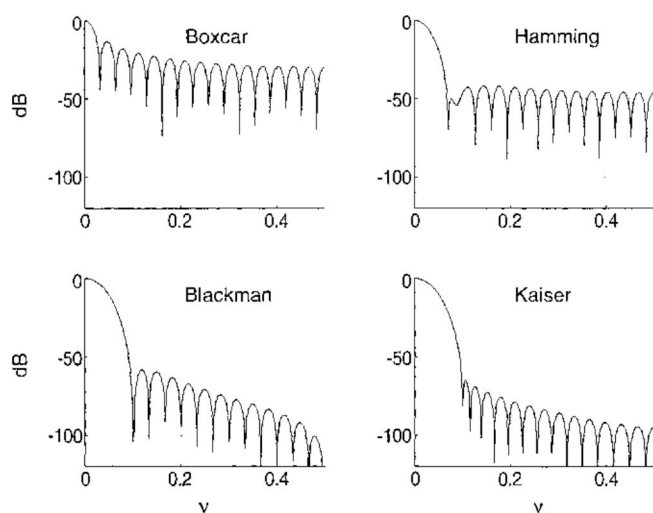


FIGURE 5.7

Continuous power gain spectra of the rectangular (boxcar) window and three other window functions, each scaled to produce 0 dB at $v = 0$; $N = 31$ weights.

TABLE 5.1

Effects of Window Spectrum Properties on FIR Filter Power Gain

Window Spectrum Property	Effect on Power Gain of FIR Filter
Wider main lobe	Wider transition from passband to stopband
Higher side lobes	Higher ripple in passband and stopband

TABLE 5.2

Window Formulas — Algebraic and MATLAB

Window	Formulas for Weights: $k = 0, 1, \dots, N-1$ (Algebraic), or $k = [0:N-1]$ (MATLAB)
Hamming	$w_k = 0.54 - 0.46 \cos\left(\frac{2\pi k}{N-1}\right)$ $w = 0.54 - 0.46 * \cos(2 * \pi * k / (N - 1))$
Blackman	$w_k = 0.42 - 0.5 \cos\left(\frac{2\pi k}{N-1}\right) + 0.08 \cos\left(\frac{4\pi k}{N-1}\right)$ $w = 0.42 - 0.5 * \cos(2 * \pi * k / (N - 1)) + 0.08 * \cos(4 * \pi * k / (N - 1))$
Kaiser	$w_k = \frac{I_0\left(\beta \sqrt{1 - \left(\frac{2k}{N-1} - 1\right)^2}\right)}{I_0(\beta)}$ $w = \text{bessel_0}(\text{beta} * \text{sqrt}(1 - (2 * k / (N - 1) - 1) .^2)) / \text{bessel_0}(\text{beta})$

Before discussing the other three windows in detail, imagine convolutions in (5.7) using the power spectra shown in Figure 5.7, convolving each of these from right to left over the ideal lowpass function in Figure 5.1 (with the gain in dB instead of amplitude units). As the main lobe at the left end of the window spectrum passes over the cutoff point of the ideal filter spectrum, the convolution causes the resulting gain to spread or “roll off” around the cutoff frequency. Then, as the window spectrum moves further to the left, the side lobes interact with the ideal spectrum to produce the ripple we have seen in the resulting passband and stopband. In brief, we may summarize these effects as in Table 5.1.

Having seen how windows operate to smooth the ripple in the FIR power gain, we turn now to formulas for the windows. The subject of windows is extensive in the literature on signal processing and statistical analysis of time series.¹⁻⁷ Here we present, besides the boxcar, just the three windows in Figure 5.6 that are well known and have certain optimal properties. Algebraic and MATLAB expressions for these windows are given in Table 5.2. The expressions are easy to understand and implement, except for the Kaiser expressions, which require explanation.

The *Hamming* window is the raised cosine function illustrated in Figure 5.6 and is usually the preferred “two-term” function in filter design. Another window, the *Hanning* window, named after J. von Hann, is similar but is zero at the end points. The *Blackman* window is a modulated version of the Hanning window. As seen in Figure 5.7, the Blackman window has a relatively wide main spectral lobe and lower side lobes, resulting in a smoother FIR power gain function with longer transition between passband and stopband.

The Kaiser window requires more discussion, but it is worth the trouble, because its properties lead to superior FIR filter designs. Unlike the other windows, it has a parameter (β), which has been set in Figures 5.6 and 5.7 to produce a main spectral lobe about the same as that of the Blackman window. Typically, the value of β is in the range from four to nine. Values of β that give main lobe widths comparable to those of the Hamming and Blackman windows are

$$\begin{aligned} \beta &= \sqrt{3}\pi && \text{Kaiser and Hamming main lobe widths about equal.} \\ \beta &= 2\sqrt{2}\pi && \text{Kaiser and Blackman main lobe widths about equal.} \end{aligned} \quad (5.8)$$

Power spectra of windows with these values of β are illustrated in Figure 5.8 with $N = 127$ weights. In the upper half of the figure, for example, with $\beta = \sqrt{3}\pi$, the Kaiser and Hamming windows have similar main lobes. However,

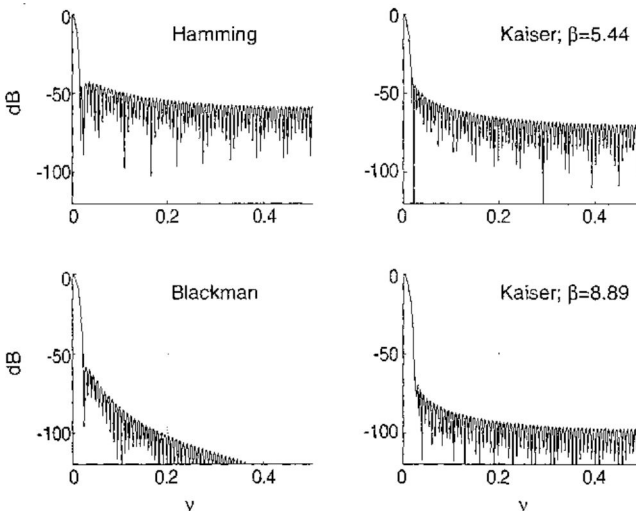


FIGURE 5.8
Power spectra of Kaiser windows with β set to give main lobe widths approximately the same as Hamming and Blackman main lobe widths.

the Kaiser window has lower side lobes, resulting in less ripple in an FIR filter gain function.

The Kaiser window as presented in Table 5.2 also requires computation of the zero-order modified Bessel function of the first kind, given by

$$I_0(x) = 1 + \sum_{n=1}^{\infty} \left(\frac{(x/2)^n}{n!} \right)^2 \quad (5.9)$$

Fortunately, this sum converges rapidly due to the factorial in the denominator and is easily approximated in various ways. The MATLAB function *besseli*(0, *x*) may be used to compute (5.9). If MATLAB is not available, a simple algorithm called *bessel_0* is included in the software. The algorithm is in the MATLAB language but would be easy to translate. The use of *bessel_0* is illustrated in Table 5.2.

There is an important point about using windows in the manner just described to design lowpass FIR filters: due to the truncation of the original ideal impulse response followed by the application of a window, the windowed weights may require *scaling* to adjust the lowpass gain to 0 dB at $\nu = 0$. (We noted that this was done for the plots in Figure 5.7.) For lowpass filters, because the amplitude gain at $\nu = 0$ is just the sum of the weights, the adjustment (in MATLAB notation) after the weights are computed is simply

$$b = b/\text{sum}(b)$$

For highpass, bandpass, and bandstop filters, the gain may be adjusted to one at $\nu = 0.5$, $(\nu_1 + \nu_2)/2$, and $\nu = 0$. These adjustments are made in the *fir_weights* function described in the next section.

To conclude this section, Figure 5.9 illustrates power gain spectra for four lowpass FIR filters, each with $N = 127$ weights and cutoff at $\nu_c = 0.1$ Hz-s. In each case, the lowpass power gain results from the use of the window with a power spectrum in the corresponding position in Figure 5.8. The relative effects on passband ripple and passband-to-stopband transition are not easy to see in Figure 5.9, but the relative effects on stopband ripple are evident.

5.5 Highpass, Bandpass, and Bandstop Filters

In the preceding section, we saw how to apply a window to a truncated ideal impulse response to achieve a lowpass FIR filter. This concept applies equally well to impulse responses of other “ideal” FIR filters with rectangular gain functions, and in particular, to highpass, bandpass, and bandstop FIR filters. Hence, all we need for these filters is the impulse response functions, or weight vectors, of the filter types to use in place of the lowpass vector, *h*, in (5.4).

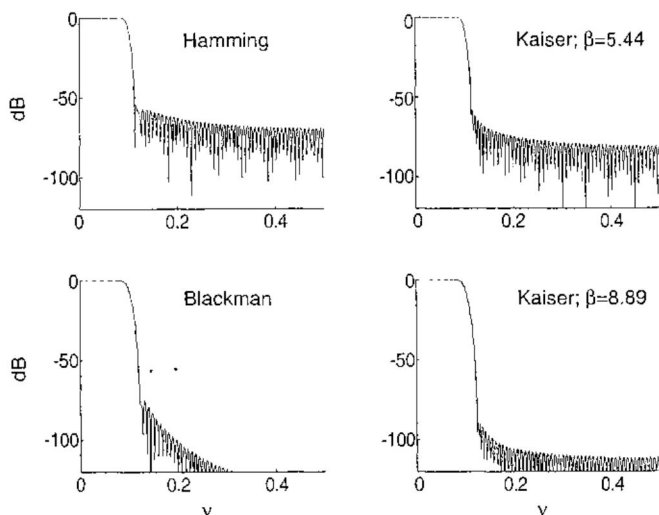


FIGURE 5.9

Lowpass FIR filter power gain; $L = 63$ and $v_c = 0.1$ Hz-s. Each power gain function results from the use of the window with corresponding power gain in Figure 5.8.

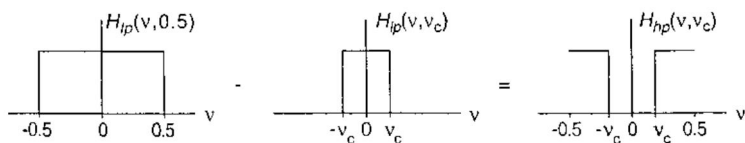


FIGURE 5.10

Allpass gain minus lowpass gain equals highpass gain, assuming the H_{ip} 's are of equal amplitude.

Weight vectors for the other filter types can be derived by subtracting lowpass weight vectors in the form of (5.4). For example, Figure 5.10 illustrates how to produce an ideal highpass filter gain by taking the difference between two ideal lowpass filter gains. We use the notation $H_{ip}(v, v_c)$ to represent the gain of an ideal lowpass filter with passband from zero to v_c Hz-s. The filter on the left in Figure 5.10 with gain $H_{ip}(v, 0.5)$ is a lowpass filter with passband extending to half the sampling frequency, that is, an *allpass* filter. The second filter is an ideal lowpass filter with cutoff at v_c Hz-s. The difference between these two gains, which are both one in the passband, is the ideal highpass gain, $H_{hp}(v, v_c)$, on the right.

Ideal bandpass and bandstop gains are produced similarly. These are denoted $H_{bp}(v, v_1, v_2)$ and $H_{bs}(v, v_1, v_2)$, respectively, where v_1 and v_2 are the

band-edge frequencies. We summarize all three conversions as follows:

$$\begin{aligned}
 &\text{Conversion of ideal lowpass FIR filter gain to} \\
 &\text{Highpass: } H_{hp}(v, v_c) = H_{lp}(v, 0.5) - H_{lp}(v, v_c) \\
 &\text{Bandpass: } H_{bp}(v, v_1, v_2) = H_{lp}(v, v_2) - H_{lp}(v, v_1) \\
 &\text{Bandstop: } H_{bs}(v, v_1, v_2) = H_{lp}(v, 0.5) - H_{bp}(v, v_1, v_2)
 \end{aligned} \tag{5.10}$$

Because the DFT is a linear operation (Table 3.3, Property 3), the implication of (5.10) is that we may construct highpass, bandpass, and bandstop FIR filters using lowpass weight vectors given by (5.4). First, we note that the impulse response of the allpass filter is the discrete impulse function. Using $v_c = 0.5$ in (5.4) and denoting the allpass weight vector b_{ap} , we have the following:

$$b_{ap} = [\text{zeros}(1, L), 1, \text{zeros}(1, L)] \tag{5.11}$$

Using similar notation, that is, using $b_{hp}(v_c)$ for the weight vector corresponding with $H_{hp}(v, v_c)$, etc., and the relationships in (5.10), the highpass, bandpass, and bandstop weight vectors may be formulated as follows:

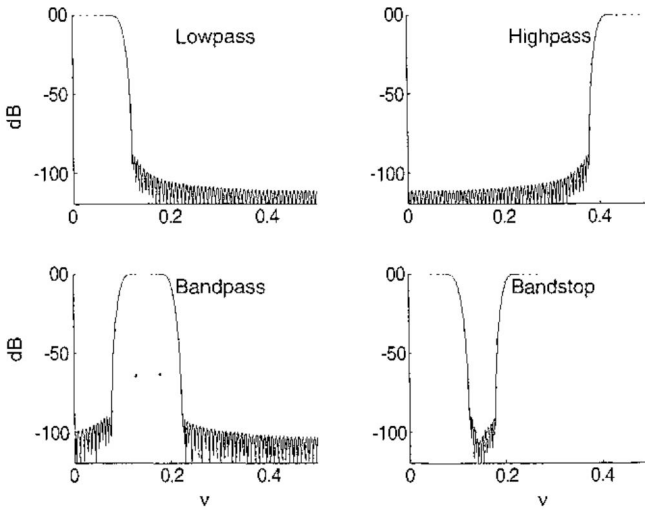
Conversion of ideal lowpass FIR filter weights to Highpass: $b_{hp}(v_c) = b_{ap} - b_{lp}(v_c)$ Bandpass: $b_{bp}(v_1, v_2) = b_{lp}(v_2) - b_{lp}(v_1)$ Bandstop: $b_{bs}(v_1, v_2) = b_{ap} - b_{bp}(v_1, v_2)$	(5.12)
---	--------

Thus, in (5.4), (5.11), and (5.12), we have weight-vector formulas for the four basic types of FIR digital filters. A MATLAB function $b = \text{fir_weights}(N, \text{band}, \text{windo}, v1, v2)$, in which N is the number of weights, *band* designates the band (lowpass, highpass, etc.), *windo* designates the window, and $[v1, v2]$ are the critical frequencies, is provided so the reader can see the equations in algorithmic form.

Any of the windows discussed in the previous section may be applied to the weight vectors in (5.12), either before or after they are combined, with the same effect. Another function, $w = \text{window}(N, \text{type}, \text{beta})$, generates several different nonnormalized windows and is used internally by *fir_weights*. For example, the weight vector for the FIR filter with $N = 127$ weights, *band* = 1 for lowpass, *windo* = [7, 8.89] for the Kaiser window with $\beta = 8.89$, and $v_c = 0.1$ Hz-s on the lower right in Figure 5.9 is produced by the following expression:

$$b = \text{fir_weights}(127, 1, [7, 8.89], .1); \tag{5.13}$$

To conclude this section, four examples of FIR filter power gain are presented in Figure 5.11. In each case, the number of weights is $N = 127$. The four

**FIGURE 5.11**

Power gain of four typical FIR filters, each with $N = 127$ weights and using the Kaiser window with $\beta = 8.89$.

windowed weight vectors were computed with the following expressions:

$$\begin{aligned}
 b_l &= \text{fir_weights}(127, 1, [7, 2\sqrt{2}\pi], .1); \\
 b_h &= \text{fir_weights}(127, 2, [7, 2\sqrt{2}\pi], .4); \\
 b_p &= \text{fir_weights}(127, 3, [7, 2\sqrt{2}\pi], .1, .2); \\
 b_s &= \text{fir_weights}(127, 4, [7, 2\sqrt{2}\pi], .1, .2);
 \end{aligned} \tag{5.14}$$

Let b stand for any of the four weight vectors above. The filter gain in each case in Figure 5.11 is as follows:

$$H = \text{fft}(b, 1000); \tag{5.15}$$

Hopefully, the reader is familiar by now with the effect of appending zeros to the windowed weight vector before taking the FFT. The result in this case is a DFT with 1000 points and a 501-point gain function in the range $[0, 0.5]$ Hz-s, that is, from zero through half the sampling rate. The final step is to convert the amplitude of H to dB:

$$\text{dB} = 20 \cdot \log_{10}(\max(\text{abs}(H(1:500)), 1.e-6)); \tag{5.16}$$

The resulting power gain in dB is plotted in Figure 5.11 for each of the four filters.

5.6 A Complete FIR Filtering Example

We now have all the tools necessary for the application of any of the four common types of FIR filters to a digitized signal vector. There are a few practical matters worth noting when this is done, such as conversion of units, phase shift, etc. To illustrate some of these, we provide the following example in which an input vector consisting of a signal with additive high-frequency random noise is filtered to improve the signal-to-noise ratio. We should note at the outset that this is a typical application of filtering, but rarely do the results look this good in practice. The reader should not be discouraged when the results do not look as good with real data.

Suppose you are looking for electromagnetic pulses emitted by a distant source known to be sending in a band between 100 and 200 MHz. A sensitive antenna is aimed at the source, and the receiver output is digitized at 10^9 samples per second. One microsecond of data (that is, 1000 samples) of the receiver output are plotted in the upper part of Figure 5.12.

Suppose also that most of the noise power is known to be concentrated at frequencies above 300 MHz. Having this information, you put the receiver output through a lowpass FIR filter cutting off at 250 MHz. The filter you choose is similar to the upper left filter in Figure 5.9. It uses the Hamming window and has $N = 127$ weights. The filter output is plotted in the lower half of Figure 5.12 and gives evidence of a single pulse at about 650 ns after the beginning of the 1- μ s interval.

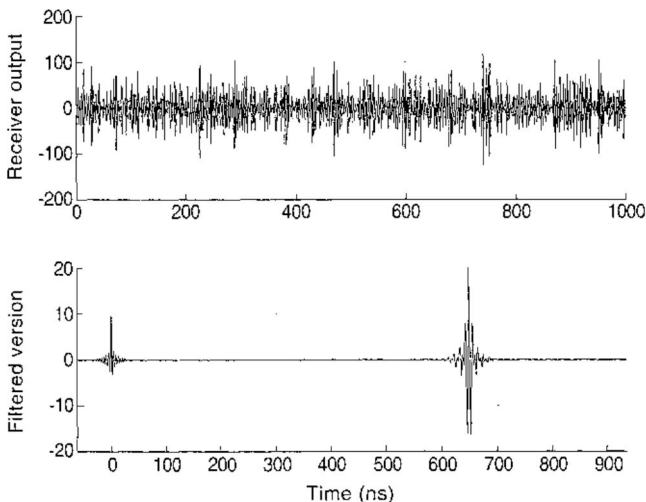


FIGURE 5.12

Receiver output consisting of a transient signal near 650 ns plus high-frequency random noise (upper plot), and filtered version (lower plot).

There are several things to note about the lower plot. First, the amplitude range is 1/10 of the upper plot range. The waveform in the vicinity of 650 ns is in exactly the same position in the upper plot but is obscured by the high-frequency noise. Second, to align the two signals, the lower plot has been shifted left 63 ns to account for the filter delay of $L = (N - 1)/2 = 63$ ns, that is, to account for the linear phase shift of the filter. [See the discussion with (5.1)]. And third, the filter output component at 0 ns looks similar to the signal at 650 ns and might be mistaken for another pulse from the source, but it is only the filter startup transient, that is, the transient response of the filter with zero initial conditions to the onset of the upper waveform.

There are other things to consider as well, such as the choice of the number of filter weights and the cutoff frequencies, etc., in addition to assuring that the filter output is being correctly interpreted.

5.7 Other Types of FIR Filters

Besides the four principal types of FIR filters (lowpass, highpass, bandpass, and bandstop), other FIR filter types have found use in various special applications. It is easy to generalize the procedure we have been using to obtain the design of an FIR filter with any specified gain characteristic. We simply begin with the specified continuous gain function, find the ideal impulse response as in (5.2), translate to a realizable version as we did in (5.4), and apply a window to the realizable version. In other words,

To find the weights of an FIR filter:

1. Specify a real continuous gain, $H(j\omega)$, in the range $-\pi/T \leq \omega \leq \pi/T$ such that $H(j\omega) = H^*(-j\omega)$
2. Ideal impulse response: $h_{ideal}(kT) = \frac{T}{2\pi} \int_{-\infty}^{\infty} H(j\omega) e^{j\omega kT} d\omega;$
 $-\infty < k < \infty$
3. Realizable, causal version: $h_k = h_{ideal}((k - L)T); \quad 0 \leq k \leq N - 1;$
 $N = 2L + 1$
4. FIR filter weights: $b_k = w_k h_k; \quad 0 \leq k \leq N - 1$

(5.17)

As an example of this procedure, we design an FIR *differentiator*, that is, a filter that differentiates the input signal. The Fourier transform of the derivative of $x(t)$ with respect to t is $j\omega X(j\omega)$, which may be proved by taking the inverse transform as follows:

$$\frac{1}{2\pi} \int_{-\infty}^{\infty} j\omega X(j\omega) e^{j\omega t} d\omega = \frac{d}{dt} \left[\frac{1}{2\pi} \int_{-\infty}^{\infty} X(j\omega) e^{j\omega t} d\omega \right] = \frac{d}{dt} (x(t)) \quad (5.18)$$

Therefore, in step 1, we specify

$$H(j\omega) = j\omega; \quad |\omega| \leq \pi/T \quad (5.19)$$

In step 2, we determine the ideal impulse response (after some effort):

$$h_{ideal}(kT) = \frac{T}{2\pi} \int_{-\pi/T}^{\pi/T} j\omega e^{jk\omega T} d\omega = \begin{cases} 0; & k = 0 \\ \frac{(-1)^k}{kT}; & |k| > 0 \end{cases} \quad (5.20)$$

In step 3, we choose a filter length, $N = 2L + 1$, and determine the realizable impulse response:

$$h_k = \begin{cases} 0; & k = L \\ \frac{(-1)^{k-L}}{(k-L)T}; & k = 0, 1, \dots, L-1, L+1, \dots, 2L \end{cases} \quad (5.21)$$

Finally, in step 4, we apply a window function to obtain the weight vector, which in MATLAB notation is $b = w .* h$. The vector length (number of weights) is the overall range of k in (5.21), that is, $N = 2L + 1$.

An example of a differentiator with $N = 63$ weights using the Hamming window is presented in Figure 5.13. The amplitude gain, $|H(j\omega)|$, is plotted

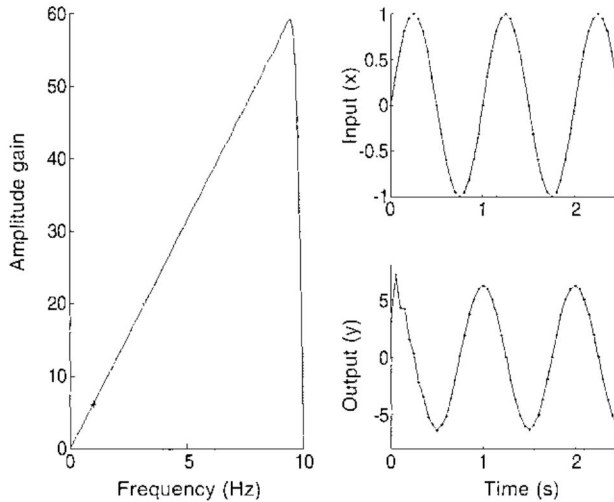


FIGURE 5.13

Amplitude gain of FIR differentiator with $N = 63$ weights and Hamming window (left). The “*” on the gain plot indicates the frequency (1 Hz) of an input unit sine wave (upper right). The differentiator output (lower right), after startup, is a cosine wave with amplitude $\omega = 2\pi$. The time step in this example is $T = 0.05$ s.

versus frequency in Hz on the left. The input signal on the upper right is a unit sine wave at 1 Hz, sampled at 20 samples/s. The output, which has been shifted left $L = 31$ samples to account for linear phase shift, is the derivative of the input with respect to time, that is, a cosine wave with amplitude equal to ω , or 2π in this case, as shown by the marker (*) on the amplitude gain plot. Notice also the startup transient on the output (y) plot, with a duration of $L = 31$ samples.

Differentiators inherently amplify high frequencies. If a signal is known to have no components above, say, ω_c , the differentiator may be designed to cut off at ω_c and thus eliminate the amplification of high-frequency noise in the output. This would be done simply by changing the limits of integration in (5.20) from $\pm\pi/T$ to $\pm\omega_c$. The result, in place of (5.21) is as follows:

$$h_k = \begin{cases} 0; & k = L \\ \frac{\omega_c \cos((k-L)\omega_c T)}{(k-L)\pi} - \frac{\sin((k-L)\omega_c T)}{(k-L)^2 \pi T}; & 1 \leq |k-L| \leq L \end{cases} \quad (5.22)$$

An example of this with $\omega_c = 4\pi$ rad/s is shown in Figure 5.14. Because the input signal at $\omega_c = 2\pi$ rad/s is entirely below ω_c , the results after startup are the same as in Figure 5.13.

Another example of the design procedure in (5.17), and another example of a filter with special use in signal processing, is the FIR *Hilbert transformer*,

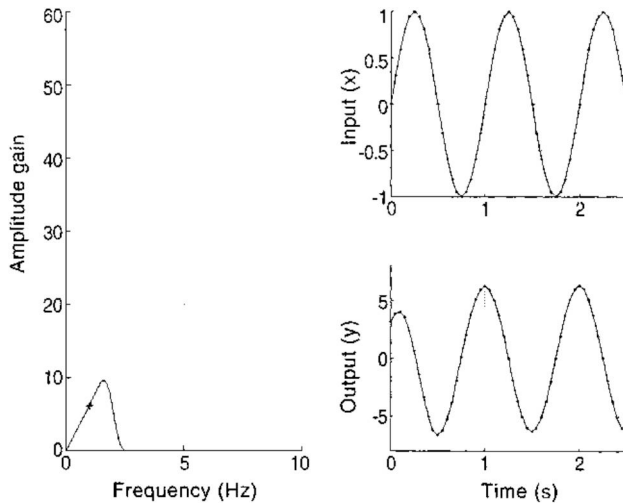


FIGURE 5.14

Repetition of Figure 5.13 with differentiator cutoff reduced from π/T to $f_c = 2$ Hz.

which is a digital filter designed ideally to provide a 90° phase shift at all frequencies below half the sampling rate. Its ideal transfer function is simply

$$H(j\omega) = \begin{cases} j; & -\pi/T < \omega < 0 \\ -j; & 0 \leq \omega < \pi/T \end{cases} \quad (5.23)$$

The Hilbert transformer is used typically with *complex* signals with samples that have real and imaginary parts. The output of a radar detector, for example, consists of real and imaginary or *quadrature* components on separate channels, and taken together, these become a complex signal with real and imaginary parts.

The operation of a Hilbert transformer on complex signals brings up an interesting spectral property of these signals. Suppose we have a complex signal vector, $s = x + jy$. Because the DFT is a linear operation, its DFT components must be $S_m = X_m + jY_m$. We recall from (3.11) and Table 3.3, Property 2, that, at negative frequencies, $X_{-m} = X'_m$ when x is real. The result is $S_{-m} = X'_m + jY'_m$ for the spectrum of a complex signal at negative frequencies. Now suppose the imaginary component, y , is the Hilbert transform of the real component, x . This implies $Y_m = -jX_m$ for $m \geq 0$ and $Y_m = jX_m$ for $m < 0$, and therefore, $S_m = X_m + j(-jX_m) = 2X_m$ and $S_{-m} = X'_m + j(jX'_m) = 0$ for $m \geq 0$. Thus, we have the following result:

If s is the complex signal $s = x + jy$, and if y is the Hilbert transform of x , then the spectrum of s is one-sided, that is,

$$S_m = \begin{cases} 2X_m; & m \geq 0 \\ 0; & m < 0 \end{cases}$$

(5.24)

The FIR Hilbert transformer is easy to design using the procedure in (5.17). The ideal impulse response in step 2 of the procedure is

$$h_{ideal}(kT) = \begin{cases} 0; & k \text{ even} \\ \frac{2}{k\pi}; & k \text{ odd} \end{cases} \quad (5.25)$$

The rest of the procedure, as well as the application of the Hilbert transformer to complex signals in communications and other related areas, is described in references 2, 3, and 9.

5.8 Exercises

General Instructions: (1) Whenever you make a plot in any of the exercises, be sure to label both axes so we can tell exactly what the units mean. (2) Make continuous plots unless otherwise specified. "Discrete plot" means to plot discrete symbols, unconnected unless otherwise specified. (3) In plots, and especially in subplots, use the *axis* function for best use of the plotting area.

1. Plot the unweighted, linear-phase weights given by (5.4) in two subplots, one above the other. Set $L = 200$ for both plots. Set the cutoff frequency (ν_c) at 0.03 Hz-s in the upper plot and 0.3 Hz-s in the lower plot.
2. Complete the following:
 - a. Derive an expression for the "upper envelope" of (5.4), that is, an expression for a smooth curve passing through all the positive peaks beyond the second peak at $k - L = 5/(4\nu_c)$.
 - b. For $\nu_c = 0.03$, how far from the peak impulse response (b_L) does this envelope drop below 5% of the peak?
 - c. Verify your answers to (a) and (b) by plotting b_k/b_L as well as the upper envelope of b_k/b_L for $240 \leq k \leq 400$. On your plot, use abscissa limits $[L, 2L]$ and ordinate limits $[0, 0.5]$.
3. Make a plot similar to Figure 5.3 of the power gains of two unwrapped FIR lowpass filters with weights given by (5.4). Use the following data:

Sampling frequency: 20 samples/s	Power gain from -60 to 5 dB
Cutoff frequency: 1 Hz	Frequency from 0 to 10 Hz
Number of weights: 33 and 129	
4. Plot the power gains of the two filters described in Exercise 1 in a single figure with no subplots. Plot dB in the range $[-80, 5]$ versus frequency in Hz-s in the range from zero to half the sampling rate.
5. Complete the following:
 - a. Show that the amplitude gain of any linear-phase filter with weights in the form of (5.3) may be expressed in the following form:

$$|H(\omega T)| = |b_0 + 2 \sum_{k=1}^L b_k \cos(\omega k T)|$$

[Hint: Use the weight vector in (5.3) centered at zero, noting that the time shift has no effect on the amplitude spectrum.]

- Compare this result with (2.32), and note that here we have a continuous Fourier series for a function of *frequency* (not time).
- b. Express the amplitude gain at half the sampling rate as a simple, nontrigonometric sum. Use this expression to compute the power gains of the two filters in Exercise 1 at half the sampling rate. If you have worked Exercise 4, use this result to verify the ends of the two dB plots.
6. Write a MATLAB function, $b = \text{my_fir_lowpass_weights}(N, n_{uc})$ that will compute unwindowed weights of any linear-phase lowpass FIR filter, where N is the number of weights, and n_{uc} is v_c , the cutoff frequency in Hz-s. Write the function so that if N is even, it will be increased by one with a warning.
 - a. Test the function with $N = 7$ and $N = 8$ to see that it prints the correct number of weights and issues a warning when $N = 8$.
 - b. Use the function to plot the unweighted impulse response of a lowpass filter with $v_c = 0.3$ and $N = 401$. Compare the plot with your answer to Exercise 1, or verify it by computing (5.2) and plotting the result. If the plots are the same, you may use the function in subsequent exercises.
 7. Write a MATLAB function called $w = \text{my_window}(N, \text{type}, \beta)$ to create w , a row vector of N window samples. Let *type* designate any of the three windows in Table 5.2 plus the boxcar window, and let *beta* designate the parameter β , for the Kaiser window. (Note: You may use the MATLAB function *nargin* to determine whether there are two or three input arguments.) Plot all four windows in a single figure for $N = 15$, using $\beta = 7$ for the Kaiser window. Make connected discrete plots using a different symbol for each window, and print a legend on your plot.
 8. Make connected discrete plots of the Hamming, Blackman, and Kaiser windows with $N = 63$ weights in three vertically stacked subplots. Make the Kaiser window look like the Blackman window. Label the plots to show which is which.
 9. Design a lowpass FIR filter to cut off at 12.5 MHZ, assuming a sampling frequency of 100 MHZ. Use the Hamming window, and use $L = 12$. Plot the power gain in terms of dB vs. frequency in MHZ.
 10. The specifications of an FIR lowpass filter are as follows:
 Time step: 100 μs ; Cutoff: 1.0 KHz; Power gain ≤ -40 dB for $f \geq 2$ KHz;
 Window: Blackman
 Design the filter by making 11 plots in the same graph of the power gain in terms of dB versus frequency in KHz using $L = [5:15]$. Indicate the smallest value of L that meets the specifications.
 11. In a single graph, plot the power gains of two lowpass FIR filters with T and v_c the same as in Exercise 10. Use the Blackman window

for one filter and the Kaiser window for the other, with β set to match the Blackman window. Use $N = 31$ weights in both filters. Plot dB in the range $[-100, 0]$ versus frequency in KHz, and identify each plot. Briefly compare the two filters.

12. Design a highpass FIR filter with 41 weights to operate with 10^6 samples/s, with passband beginning at 100 KHz. Use the Kaiser window with $\beta = 6$. Plot power gain in dB in the range $[-100, 0]$ versus frequency in KHz. What is the *amplitude gain* at cutoff?
13. Design a bandpass FIR filter with $L = 30$ to operate with time step $T = 0.1$ s and a passband from 3 to 4 Hz. Plot power gain in dB in the range $[-100, 0]$ versus frequency in Hz.
14. Complete the following:
 - a. Create a Gaussian noise vector using the following expressions:

```
N = 200; randn('state',0); x = 10*randn(1,N);
```

Treat this vector as a waveform sampled at 100 samples/s and beginning at $t = 0$. Next, create vector y by passing x through a lowpass FIR filter designed with 61 weights, the Hamming window, and cutoff at 30 Hz. Finally, add to y a unit 40-Hz sine wave beginning at $t = 1.0$ s and extending to the end of y . Plot y versus t in the upper of two subplots.

- b. Create vector u by passing y through a highpass FIR filter with 61 weights, the Hamming window, and cutoff at 35 Hz. Plot u versus t in the lower subplot, and explain the result.
15. Complete the following:
 - a. Create vector x consisting of 200 samples of the sum of five unit sine waves with frequencies 10, 15, 20, 25, and 30 Hz, using time step $T = 10$ ms. (For full credit, create x with a single MATLAB expression.) Plot x versus t in seconds in the upper of two subplots.
 - b. Design a bandpass filter with 63 weights to select the component of x at 15 Hz. Use one of the windows in Table 5.2. Plot the filtered version of x in the lower subplot, and comment on the effectiveness of the filter.
16. Complete the following:
 - a. Create the vector x in Exercise 15 with $N = 300$ samples. Plot $x(t)$ in the range $t = [0, 2]$ s in the first of three subplots.
 - b. Filter x as in Exercise 15, but this time with a bandstop filter to remove the component at 15 Hz. Plot the filter output in the center subplot.
 - c. Shift the filter output to account for the delay caused by the filter. Then subtract it from x . Plot the result in the lower subplot

- over the same range of t . What is the frequency of the final waveform?
17. Derive the impulse response of the ideal differentiator, beginning with $H(j\omega) = j\omega$ and using integration by parts.
 18. Complete the following:
 - a. Create the vector $p = [0:2 * T:1, 1 - T:-T:0]$. Let x consist of three cycles of p . Make a continuous plot of $x(t)$ versus t using $T = 0.01$ s in the upper of three subplots.
 - b. In the center subplot, make a connected discrete plot of the impulse response of a differentiator using $L = 10$ and the Hamming window. Plot the response vs. time t .
 - c. In the lower subplot, show the differentiator output when the input is x . Shift the output so it is correctly aligned with x in the upper subplot.
 19. Make two subplots. On the left, plot $N = 101$ samples of the sum of the impulse responses of an ideal allpass filter with gain equal to 2.0 and an ideal lowpass filter with gain equal to 1.0 and $\nu_c = 0.2$ Hz-s. On the right, plot the amplitude gain of this combined filter after applying the Hamming window.

References

1. Harris, F.J., On the use of windows for harmonic analysis with the discrete Fourier transform, *Proc. IEEE*, 66, 51, Jan. 1978.
2. Oppenheim, A.V. and Schaffer, R.W., *Discrete-Time Signal Processing*, Prentice Hall, Englewood Cliffs, NJ, 1989, chap. 7.
3. Orfanidis, S.J., *Introduction to Digital Signal Processing*, Prentice Hall, Upper Saddle River, NJ, 1996, chap. 10.
4. Kaiser, J.F., Design methods for sampled data filters, *Proc. First Allerton Conf. on Circuit and System Theory*, 221, Nov. 1963.
5. Kaiser, J.F., Digital filters, in *System Analysis by Digital Computer*, Kuo, F.F. and Kaiser, J.F., Eds., John Wiley & Sons, Inc., New York, 1966, chap. 7.
6. Blackman, R.B. and Tukey, J.W., *The Measurement of Power Spectra*, Dover Publications, New York, 1958.
7. Helms, H.D., Nonrecursive digital filters: design methods for achieving specifications on frequency response, *IEEE Trans. Audio and Electroacoust.*, 16, 336, Sept. 1968.
8. Mitra, S.K. and Kaiser, J.F., Eds., *Handbook of Digital Signal Processing*, Wiley, New York, 1993.
9. Parks, T.W. and Burrus, C.S., *Digital Filter Design*, Wiley, New York, 1987.
10. Antoniou, A., *Digital Filters: Analysis and Design*, 2nd ed., McGraw-Hill, New York, 1993.
11. Elliott, D.F., Ed., *Handbook of Digital Signal Processing*, Academic Press, New York, 1987.

12. Rabiner, L.R. and Rader, C.M., Eds., *Digital Signal Processing*, IEEE Press, New York, 1972.
13. Roberts, R.A. and Mullis, C.T., *Digital Signal Processing*, Addison-Wesley, Reading, MA, 1987.
14. Jackson, L.B., *Digital Filters and Signal Processing*, Kluwer Academic Publishers, Norwell, MA, 1989.
15. Lutovac, M.D., Tomic, D.V., and Evans, B.L., *Filter Design for Signal Processing Using MATLAB and Mathematica*, Prentice Hall, Upper Saddle River, NJ, 2001.

6

IIR Filter Design

6.1 Introduction

In Chapter 4, we saw that the transfer function of a linear recursive or IIR filter is a ratio of polynomials in z , rather than just a single polynomial that describes an FIR filter. The direct-form algorithm, transfer function, and gain of an IIR filter was given in (4.10) through (4.12):

Algorithm:	$y_k = \sum_{n=0}^{N-1} b_n x_{k-n} - \sum_{m=1}^{M-1} a_m y_{k-m}$	
Transfer function:	$H(z) = \frac{Y(z)}{X(z)} = \frac{b_0 + b_1 z^{-1} + \cdots + b_{N-1} z^{-(N-1)}}{1 + a_1 z^{-1} + \cdots + a_{M-1} z^{-(M-1)}}$	(6.1)
Filter gain:	$H(e^{j\omega T}) = \frac{b_0 + b_1 e^{-j\omega T} + \cdots + b_{N-1} e^{-j(N-1)\omega T}}{1 + a_1 e^{-j\omega T} + \cdots + a_{M-1} e^{-j(M-1)\omega T}}$	

Thus, the IIR transfer function has poles as well as (usually) zeros on the z -plane, and, as stated in (4.27), the poles must be inside the unit circle for stability. As we shall see in the next section, this restriction on the location of the poles results in the impossibility of a stable, linear-phase IIR filter operating in real time.

On the other hand, as noted in Section 5.1, the presence of poles in $H(z)$ away from the origin can also result in a sharper transition from the passband to the stopband of an IIR filter, compared with an FIR filter with the same total number of weights. With this advantage, IIR filters are still used in situations where at least a small amount of phase distortion is allowable.

As in Chapter 5 for FIR filters, the primary purpose of this chapter is to develop weight-vector formulas for the standard designs of lowpass, high-pass, bandpass, and bandstop IIR filters. Our approach will also be similar to the development in Chapter 5, that is, beginning with an "ideal" lowpass design, creating from this a realizable IIR lowpass filter, and finally transforming the latter into the other three filter types.

6.2 Linear Phase

As we saw in Chapter 5, the key to understanding linear phase is DFT Property 8 in Table 3.3, that is, a uniform shift in time transforms to a phase angle which is a linear function of frequency. This, in turn, led to the conclusion in (5.1), that is, if $H(e^{j\omega T})$ is the gain function of a linear-phase filter, the following condition must hold:

$$H(e^{j\omega T}) = [H(e^{j\omega T})]^* e^{-jn\omega T} = R(e^{j\omega T}) e^{-jn\omega T} \quad (6.2)$$

where $R(e^{j\omega T})$ is a real function of ω and therefore equal to its own conjugate, $R(e^{-j\omega T})$. In terms of the z -transform, (6.2) becomes

$$H(z) = z^{-n} R(z); \quad R(z) = R(z^{-1}) \quad (6.3)$$

Now suppose $R(z)$ is a ratio of polynomials in z , as it must be for an IIR filter. Then, if $R(z)$ has a pole at z_0 , the condition in (6.3) implies $R(z)$ must also have a pole at $z^{-1} = z_0$, that is, a pole at $z = 1/z_0$. Thus, $R(z)$ and therefore $H(z)$ cannot be stable, because if z_0 is inside the unit circle, $1/z_0$ must be outside the unit circle. Thus, any stable IIR filter in the form of (6.1) cannot have a linear phase characteristic.

Even though linear phase is not possible in (6.1), there are two points worth noting. First, the filter designs introduced in this chapter have, as we shall see, phase characteristics that are nearly linear through most of the passband. The stopband phase is usually not of concern, and so the nearly linear phase in portions of the passband results in many useful applications of IIR digital filters.

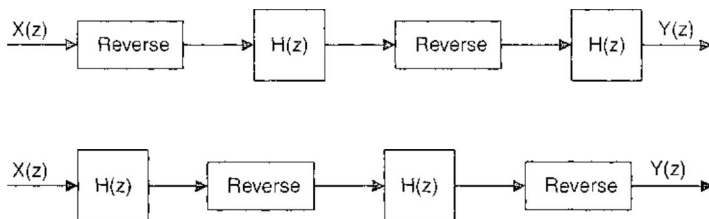
Second, if a digital filter (IIR or FIR) is not operating in real time, there is a way to operate the filter without phase shift, that is, a way to modify the operation of the filter so that the resulting gain is real. The method involves reversing the input or the output signal vectors.

In (4.6), we defined $X(z)$ as the z -transform of a signal vector x , that is, the z -transform of a sequence of samples beginning at $t = 0$ and ending at $t = (N - 1)T$. Now suppose we create a vector x^r by reversing this sequence so that it begins at $t = 0$ and ends at $t = -(N - 1)T$. Obviously, we could not reverse x in real time, but if x were recorded, we could reverse the elements of the signal vector. Thus,

$$x = [x_0, \dots, x_{N-1}]; \quad x^r = [x_{-(N-1)}, \dots, x_0] \quad (6.4)$$

To transform x^r , we must extend the transform definition in (4.6) to include negative index values; that is,

$$X^r(z) = \sum_{n=-(N-1)}^0 x_n z^{-n} = \sum_{m=0}^{N-1} x_{-m} z^m = X(z^{-1}) \quad (6.5)$$

**FIGURE 6.1**

Filtering without phase shift via time series reversal. In both cases, the overall gain is real and equal to $|H(e^{j\omega T})|^2$.

Thus, reversing a signal vector is equivalent to substituting z for z^{-1} in its z -transform. An obvious corollary to this result is that reversal is its own inverse, that is, $\{X'(z)\}^r = X(z)$. Furthermore, reversal is distributive, that is, $\{X(z)H(z)\}^r = X'(z)H'(z)$.

Now consider the two equivalent operations in Figure 6.1. In each case, the "Reverse" labels mean that the output of the box is the reversed input vector. When we apply (6.5) and its corollaries to either of the two operations, the result is the same:

$$\begin{aligned} Y(z) &= \{X'(z)H(z)\}^r H(z) = X(z)H(z^{-1})H(z) = X(z)|H(z)|^2 \\ Y(z) &= \{\{X(z)H(z)\}^r H(z)\}^r = X(z)H(z)H(z^{-1}) = X(z)|H(z)|^2 \end{aligned} \quad (6.6)$$

Thus, the overall gain in Figure 6.1, $|H(e^{j\omega T})|^2$, is real and equal to the squared magnitude of the gain of the filter used twice in each operation. In this limited sense, linear and zero phase shift are realizable with all digital filters, including IIR filters.

6.3 Butterworth Filters

In this and the next section, we introduce two of the most commonly used IIR filters, the Butterworth and Chebyshev filters. These filters were invented in the form of continuous filters long before the advent of DSP. Each is optimal in a certain sense. Our purpose here is first to describe the lowpass version of each type of filter and then to show how to convert these to digital lowpass IIR filters that are optimal in the same sense.

We begin with the Butterworth filter. The power gain of the ideal, continuous, lowpass version of this filter is given by

$$|H(j\omega)|^2 = \frac{1}{1 + (j\omega/j\omega_c)^{2L}} \quad (6.7)$$

where ω is frequency in rad/s, and ω_c is the cutoff frequency at the end of the passband, also in rad/s. The maximum power gain of this ideal filter is $|H(j0)|^2 = 1$, and the power gain at cutoff is by definition $|H(j\omega_c)|^2 = 1/2$ (unlike the FIR filters, with *amplitude gain* = 1/2 at cutoff).

The filter is named for S. Butterworth.¹⁴ Butterworth was able to show that, of all possible rational functions of ω of order L , (6.7) has “maximum flatness” in the passband, that is, a maximum number of vanishing derivatives of the power gain function at $\omega = 0$, and is thus optimal in this sense. Using $\omega_c = 1$, examples of (6.7) for the first three even values of L , each illustrating the maximum flatness property, are shown in Figure 6.2.

When we use s in place of $j\omega$ to get the Laplace transfer function, (6.7) takes one of two forms depending on whether L , the order of the filter, is odd or even. To simplify (6.7) as well as the rest of our discussion, we will assume L is even at this point. With this assumption, we have

$$|H(s)|^2 = \frac{1}{1 + (s/j\omega_c)^{2L}} = \frac{1}{1 + (s/\omega_c)^{2L}}; \quad L \text{ even} \quad (6.8)$$

A unique set of poles of this expression on the s -plane is given by

$$s = \omega_c(-1)^{\frac{1}{2L}} = \omega_c(e^{j(2n-1)\pi})^{\frac{1}{2L}} = \omega_c e^{j\frac{(2n-1)\pi}{2L}}; \quad n = 1, 2, \dots, 2L \quad (6.9)$$

These are spaced evenly around the circle with radius ω_c on the s -plane. The placement of poles for the first three even values of L , corresponding with the gain curves in Figure 6.2, is shown in Figure 6.3.

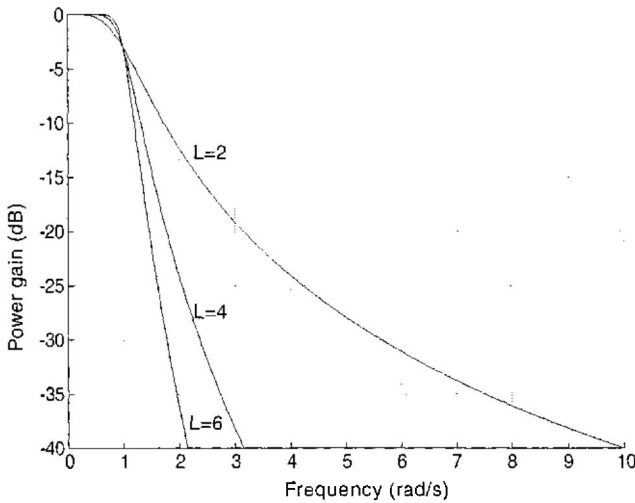


FIGURE 6.2

Butterworth power gain curves with $\omega_c = 1$ for the first three even values of L .

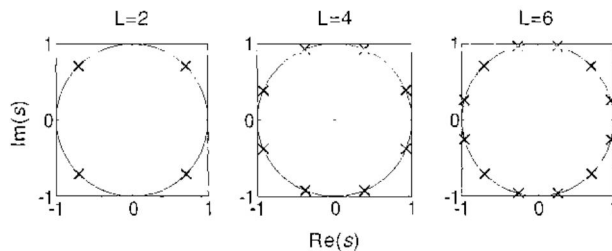


FIGURE 6.3

Butterworth power gain poles on the s -plane for the first three even values of L ; $\omega_c = 1$.

For continuous filters, we will see shortly that the stability requirement for continuous systems implies that the s -plane poles must all be to the left of the imaginary axis, that is, on the left half-plane. Because $|H(s)|^2$ in (6.8) is equal to $H(s)H(-s)$, we may therefore specify the stable Butterworth transfer function by using only the left half-plane poles in (6.9), that is, poles with angles that lie in the range

$$\frac{\pi}{2} < \frac{(2n-1)\pi}{2L} < \frac{3\pi}{2}, \quad \text{or} \quad L < 2n-1 < 3L \quad (6.10)$$

Because we have assumed L is even, this means that the L values of n used in (6.9) to give the left-plane poles of $H(s)$ must be given by

$$n = \frac{L}{2} + 1, \frac{L}{2} + 2, \dots, \frac{3L}{2}, \quad \text{or} \quad m = n - \frac{L}{2} = 1, 2, \dots, L \quad (6.11)$$

Thus, using m in place of n in (6.9), the poles of the stable lowpass Butterworth filter are at

$$s_m = \omega_c e^{j \frac{(2m+L-1)\pi}{2L}}; \quad m = 1, 2, \dots, L \quad (6.12)$$

For $L = 2$, for example, the pole angles of $H(s)$ are $3\pi/4$ and $5\pi/4$, in agreement with Figure 6.3.

When L is even, as we have been assuming, we can see that the poles of $H(s)$ are in conjugate pairs; that is, poles s_m and s_{L+1-m} are conjugates. Therefore, we can express $H(s)$ in cascade form as follows:

$$H(s) = H_1(s) \cdot H_2(s) \cdot \dots \cdot H_{L/2}(s); \quad (6.13)$$

in which the transfer function of each section, normalized so that $H(0) = 1$, is given by

$$H_m(s) = \frac{s_m}{s - s_m} \cdot \frac{s'_m}{s - s'_m} = \frac{|s_m|^2}{s^2 - (s_m + s'_m)s + s_m s'_m}$$

$$= \frac{\omega_c^2}{s^2 - 2\omega_c \left(\cos\left(\frac{(2m + L - 1)\pi}{2L}\right) \right) s + \omega_c^2}; \quad m = 1, 2, \dots, \frac{L}{2} \quad (6.14)$$

Thus, in (6.13) and (6.14), we have the transfer function of a continuous lowpass Butterworth filter of order L with cutoff at ω_c rad/s. We will see that this form with $\omega_c = 1$ is convenient when we transform $H(s)$ to construct an equivalent digital Butterworth filter.

As we noticed in Figure 6.2, the Butterworth power gain curve becomes more rectangular as L increases. It is not difficult, in fact, to determine L based on this sort of criterion. The traditional parameters described in the literature¹⁻¹¹ specify two power gain limits. A simplified method that assumes the power gain is 0.5 (−3 dB) at the cutoff frequency, ω_c , is based on one's choice of the parameter λ , which is illustrated in Figure 6.4. Just as the passband is defined to end at ω_c , the *stopband* is defined to begin at ω_s , where the power gain of the lowpass filter is specified to be at or below $1/(1 + \lambda^2)$ at all frequencies above ω_s . The steepness (and hence the rectangularity) of

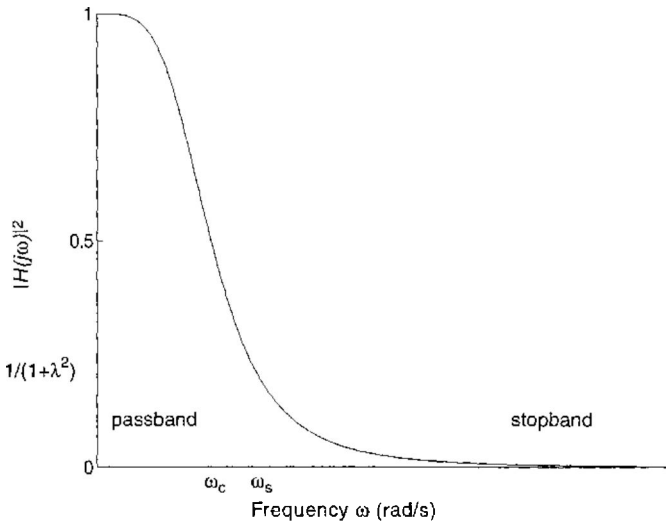


FIGURE 6.4

Passband and stopband specifications in terms of power gain.

the gain curve then depends on the interval from ω_c to ω_s , and, when this interval is specified, on one's choice of λ . Larger values of λ result in greater steepness.

If we now substitute ω_s for ω in (6.7) and use the specifications just described for the stopband, the result is

$$\frac{1}{1 + \lambda^2} = \frac{1}{1 + (j\omega_s/j\omega_c)^{2L}}, \quad \text{or} \quad L = \frac{\log_{10} \lambda}{\log_{10}(\omega_s/\omega_c)} \quad (6.15)$$

Furthermore, practical values of λ are usually 10 or more, so we may approximate the maximum stopband gain with $1/(1 + \lambda^2) \approx 1/\lambda^2$, that is, $-20 \log_{10} \lambda$ dB. Using this approximation in (6.15), we can specify the filter order (L), that is, the number of poles, needed to meet given passband and stopband specifications:

Necessary number of lowpass analog Butterworth poles:

$$L \geq \frac{(\text{maximum stopband gain in dB})}{-20 \log_{10}(\omega_s/\omega_c)}$$

(6.16)

6.4 Chebyshev Filters

Chebyshev filters are named for P. L. Chebyshev, who discovered a unique set of polynomials that may be described in either of two ways:¹³

$$\begin{aligned} V_0(x) &= 1 \\ V_1(x) &= x \\ &\vdots \\ V_L(x) &= 2xV_{L-1}(x) - V_{L-2}(x); L \geq 2 \end{aligned} \quad \begin{aligned} V_L(x) &= \cos(L \cos^{-1} x) \\ &= \cosh(L \cosh^{-1} x) \end{aligned} \quad (6.17)$$

The equivalence of these two forms of $V_L(x)$ may be shown¹³ by using the series expansions for $\cos(L\theta)$ and $\cosh(L\theta)$ in the right-hand formulas.

Chebyshev was able to show that, of all polynomials of degree L with leading coefficient equal to one, the polynomial $V_L(x)$ has the smallest maximum magnitude in the interval $|x| \leq 1$. This magnitude is, in fact, one. It happens that there is a simple way to map the interval $|x| \leq 1$ to either the passband or the stopband of the Butterworth power gain function, and thereby create a power gain curve with this maximum-magnitude or *equal-ripple* property in either of these two bands. Without going into detail, the

result for the two types of lowpass Chebyshev filters, each of which is a simple modification of (6.7), is

$$\text{Type 1: } |H(j\omega)|^2 = \frac{1}{1 + \epsilon^2 V_L^2(j\omega/j\omega_c)} \quad (6.18)$$

$$\text{Type 2: } |H(j\omega)|^2 = \frac{1}{1 + V_L^2(j\omega_s/j\omega_c)/V_L^2(j\omega_s/j\omega)} \quad (6.19)$$

The equal-ripple property of the Chebyshev polynomial is translated to the *passband* in the type 1 filter and into the *stopband* in the type 2 filter. In type 1, ϵ determines the amplitude of the passband ripple. In type 2, as for the Butterworth filter, the power gain is by definition equal to 1/2 at ω_c rad/s, and there are no parameters other than ω_c and ω_s .

The varying passband gain of the type 1 filter is undesirable in most applications, and thus, the type 2 filter is usually chosen. We will therefore focus on the design of the type 2 filter, calling it simply the *Chebyshev filter* from here on, and beginning with the derivation of its poles. (The derivation of the type 1 poles is similar and actually easier.)

The s-plane poles of the Chebyshev filter are similar to the Butterworth poles, but they are located on an ellipse rather than a circle. To find the poles, it is convenient to substitute $s = -\omega_c \omega_c / \sigma$ into (6.19), thus transforming from the s-plane to the σ -plane, and also to define the constant $\zeta = 1/V_L(\omega_s/\omega_c)$. With these definitions, (6.19) may be written

$$|H(j\omega)|^2 = \frac{\zeta^2 V_L^2(\sigma/j\omega_c)}{1 + \zeta^2 V_L^2(\sigma/j\omega_c)} \quad (6.20)$$

To derive the pole locations on the σ -plane, we set the denominator in (6.20) equal to zero and substitute the "cosine" definition in (6.17). The result is

$$V_L(\sigma/j\omega_c) = \cos(L \cos^{-1}(\sigma/j\omega_c)) = \pm j/\zeta \quad (6.21)$$

Now the inner term, $\cos^{-1}(\sigma/j\omega_c)$, is, in general, complex. Let us represent this term with $\beta + j\alpha$ and then solve for σ :

$$\begin{aligned} \cos^{-1}(\sigma/j\omega_c) &= \beta + j\alpha; \\ \sigma &= j\omega_c \cos(\beta + j\alpha) \\ &= \omega_c (\sinh \alpha \sin \beta + j \cosh \alpha \cos \beta) \end{aligned} \quad (6.22)$$

If we substitute the first line of (6.22) into (6.21), we will be able to solve for α and β , and thus for the pole locations in terms of σ :

$$V_L(\sigma/j\omega_c) = \cos(L(\beta + j\alpha)) = \cos L\beta \cosh L\alpha - j \sin L\beta \sinh L\alpha = \pm j/\zeta \quad (6.23)$$

Because the real part of this expression is zero, we must have $\cos L\beta = 0$, and therefore,

$$\beta_n = \frac{(2n-1)\pi}{2L}; \quad n = 1, 2, \dots, 2L \quad (6.24)$$

Note that these values of β are the same as the Butterworth pole angles in (6.9). Next, using these values of β and equating the imaginary components in (6.23), we have $\sin L\beta = \pm 1$, and therefore,

$$\sinh L\alpha = \pm 1/\zeta, \quad \text{or} \quad \alpha = \pm \frac{1}{L} \sinh^{-1}(1/\zeta) \quad (6.25)$$

Using (6.24) and (6.25) in (6.22), and switching to index m in a manner similar to (6.11), we can express the *right*-half σ -plane poles of the lowpass Chebyshev filter as follows:

$$\begin{aligned} \sigma_m &= \omega_c (\sinh \alpha \cos \beta_n + j \cosh \alpha \sin \beta_n); \\ \alpha &= \frac{1}{L} \sinh^{-1} \frac{1}{\zeta}; \quad \beta_m = \frac{(2m-L-1)\pi}{2L}; \quad m = 1, 2, \dots, L \end{aligned} \quad (6.26)$$

Now we may return to the s -plane, again using $s = -\omega_c \omega_c / \sigma$ and noting that the minus sign will map the right half of the σ -plane to the left half of the s -plane. Thus, we have a complete definition of the Chebyshev poles. The poles of the power gain function, that is, with $m = 1$ through $2L$, are plotted in Figure 6.5 with, $\omega_c = 1$, $\omega_s = 2$, and $L = 2, 4$, and 6 for comparison with the Butterworth poles in Figure 6.3. In this case, the poles are on an ellipse with a shape that depends on ω_c and ω_s .

The power gain function (6.20) also has zeros on the σ -plane that must be mapped to the s -plane. These are easier than the poles to find, because the inverse cosine is not complex. Setting the numerator of (6.20) equal to zero, we have

$$V_L(\sigma/j\omega_c) = \cos(L\cos^{-1}(\sigma/j\omega_c)) = 0 \quad (6.27)$$

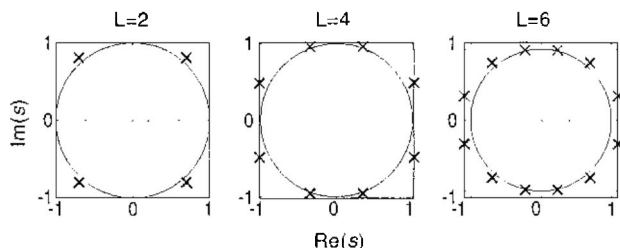


FIGURE 6.5

Chebyshev power gain poles on the s -plane with $\omega_c = 1$ and $\omega_s = 2$ for the first three even values of L .

In this case, the cos argument may be any odd multiple of $\pi/2$. The result is the following expression for the roots of (6.27), all of which are on the imaginary axis of the σ -plane:

$$\sigma_m = j\omega_c \cos\left(\frac{(2n-1)\pi}{2L}\right); \quad n = 1, 2, \dots, L \quad (6.28)$$

As before, we transform these using $s = -\omega_s \omega_c / \sigma$, and thus obtain the zeros of the lowpass Chebyshev filter, which, we note, are all on the imaginary axis of the s -plane.

Thus, beginning with (6.19) and ending with (6.28), we have derived the poles and zeros of the type 2 Chebyshev analog lowpass filter. Although the derivation is complicated compared with the Butterworth derivation, the result is easily summarized in MATLAB language:

```
% Poles.
zeta=1/cosh(L*acosh(ws/wc));
alpha=(1/L)*asinh(1/zeta);
beta=(2*(1:L)-L-1)*pi/(2*L);
sigma=wc*(sinh(alpha)*cos(beta)+j*cosh(alpha)*sin(beta));
poles=-wc*ws./sigma;
% Zeros.
sigma=j*wc*cos((2*(1:L)-1)*pi/(2*L));
zeros=-wc*wc./sigma;
```

(6.29)

Using these expressions, which locate poles and zeros in the left-half plane, all poles for $\omega_c = 1$, $\omega_s = 2$, and $L = 2, 4$, and 6 may be calculated for the plot in Figure 6.5. The zeros, which are on the imaginary axis above and below the unit circle, are not plotted in Figure 6.5.

When L is even, as we assumed for the Butterworth filter, the poles and zeros are in complex pairs, and the lowpass Chebyshev transfer function may be expressed in cascade form as follows, in a manner similar to (6.13) and (6.14), in which p_m and z_m are the poles and zeros just described.

$$H(s) = H_1(s) \cdot H_2(s) \cdots H_{L/2}(s);$$

$$H_m(s) = \frac{p_m(s - z_m)}{z_m(s - p_m)} \cdot \frac{p'_m(s - z'_m)}{z'_m(s - p'_m)} \quad (6.30)$$

$$= \frac{|p_m|^2(s^2 + |z_m|^2)}{|z_m|^2(s^2 - 2s\text{Re}\{p_m\} + |p_m|^2)}; \quad m = 1, 2, \dots, \frac{L}{2}$$

Note that the transfer function is normalized in this case so that $H_m(0)$, and therefore $H(0)$, is equal to one, that is, the filter gain is one at $\omega = 0$.

On account of the stopband ripple, it is important to have a formula like (6.16) for the Chebyshev filter so the maximum stopband gain may be specified. Using the same parameter, λ , illustrated in Figure 6.4, we can write a Chebyshev power gain expression similar to (6.15):

$$\frac{1}{1 + \lambda^2} = \frac{1}{1 + V_L^2(j\omega_s/j\omega_c)/V_L^2(1)} \quad (6.31)$$

Noting from (6.17) that $V_L^2(1) = 1$ and using the “cosh” representation of $V_L(j\omega_s/j\omega_c)$, we conclude

$$\lambda = V_L(\omega_s/\omega_c) = \cosh(L \cosh^{-1}(\omega_s/\omega_c)), \quad \text{or} \quad L = \frac{\cosh^{-1}\lambda}{\cosh^{-1}(\omega_s/\omega_c)} \quad (6.32)$$

To manipulate this into a rule like (6.16), we may use the identity $\cosh^{-1}\lambda = \log_e(\lambda + \sqrt{\lambda^2 - 1})$ with stopband gain $\text{dB} = 10 \log_{10}(1/(1 + \lambda^2))$. If we again assume λ is large (say 10 or more), these become $\cosh^{-1}\lambda = \log_e(2\lambda)$ and $\text{dB} \approx -20 \log_{10}\lambda$; that is,

$$\cosh^{-1}\lambda = \frac{\log_{10}2 + \log_{10}\lambda}{\log_{10}e} \approx \frac{\log_{10}2 - \text{dB}/20}{\log_{10}e} \approx \frac{6.02 - \text{dB}}{8.69} \quad (6.33)$$

With this approximation in (6.32), we approximate the required number of Chebyshev poles:

Necessary number of lowpass analog Chebyshev poles:

$$L \geq \frac{6.02 - (\text{maximum stopband gain in dB})}{8.69 \cosh^{-1}(\omega_s/\omega_c)} \quad (6.34)$$

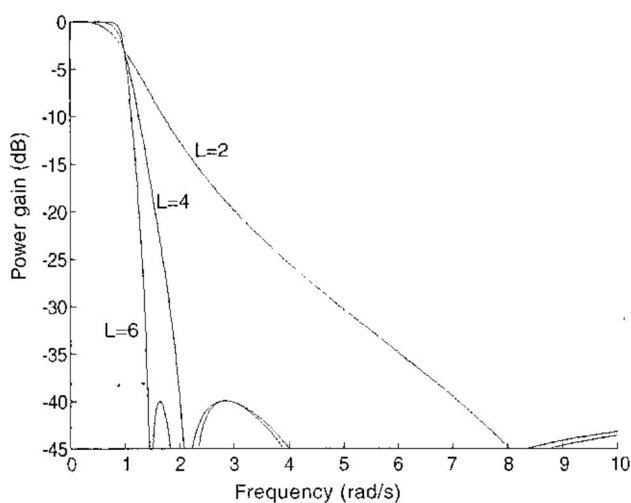
There are times when one may need to solve this expression for ω_s —for example, when the poles and zeros are found using (6.29) for a fixed value of L —in order to have a specified stopband gain. This is easily done and the result is

$$\omega_s = \omega_c \cosh\left(\frac{6.02 - (\text{maximum stopband gain in dB})}{8.69L}\right) \quad (6.35)$$

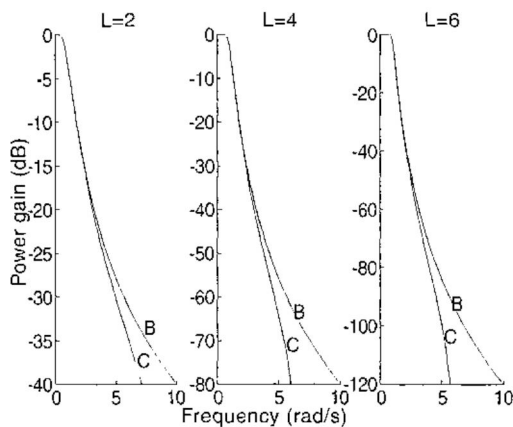
This result was used in Figure 6.6 to plot lowpass Chebyshev power gain curves for comparison with the Butterworth curves in Figure 6.2. In each case, the maximum stopband gain was set to -40 dB, and ω_s was then determined with (6.35) and used in (6.19) to find the power gain.

Another comparison of Chebyshev and Butterworth gains is shown in Figure 6.7. Here, the maximum stopband gain was set to the bottom of the plot, and in each case, we can note that the Chebyshev gain is more rectangular.

With the analog lowpass Butterworth and Chebyshev filters described, our next steps are to describe the translation from lowpass to other analog passbands, and then the conversion to digital filters with optimal characteristics.

**FIGURE 6.6**

Chebyshev power gain curves with $\omega_c = 1$ for the first three even values of L . In each case, the maximum stopband gain was set to -40 dB and (6.35) was used to find ω_s .

**FIGURE 6.7**

Comparison of Butterworth (B) and Chebyshev (C) power gain characteristics for the first three even values of L .

6.5 Frequency Translations

In the previous chapter, we transformed lowpass FIR filters into highpass, bandpass, and bandstop filters by adding and subtracting lowpass impulse response vectors to obtain the appropriate weight vectors. To transform the analog lowpass filters in this chapter, we use a different technique that accomplishes the transformation in the frequency domain.

The three frequency translations from lowpass to highpass, bandpass, and bandstop are accomplished by substituting three different functions of s for s in the lowpass Butterworth or Chebyshev transfer function, $H(s)$. The three substitutions for s , along with the corresponding substitutions for ω (found by setting $s = j\omega$) are as follows:

Lowpass to highpass:

$$s \leftarrow \frac{\omega_c^2}{s}; \quad \omega \leftarrow \frac{-\omega_c^2}{\omega} \quad (6.36)$$

Lowpass to bandpass:

$$s \leftarrow \frac{s^2 + \omega_1 \omega_2}{s}; \quad \omega \leftarrow \frac{\omega^2 - \omega_1 \omega_2}{\omega} \quad \text{with} \quad \omega_c = \omega_2 - \omega_1 \quad (6.37)$$

Lowpass to bandstop:

$$s \leftarrow \frac{s \omega_c^2}{s^2 + \omega_1 \omega_2}; \quad \omega \leftarrow \frac{\omega \omega_c^2}{\omega_1 \omega_2 - \omega^2} \quad \text{with} \quad \omega_c = \omega_2 - \omega_1 \quad (6.38)$$

Each of these is illustrated in Figure 6.8 using ideal amplitude gain functions. In each case, without much effort, you can see how the substitutions (6.36) through (6.38) translate the lowpass passband from $-\omega_c$ to ω_c into the passbands of the transformed gain functions. For example, the lowpass frequency sequence from $-\omega_c$ to 0 to ω_c is translated to the highpass range from ω_c to ∞ to $-\infty$ to $-\omega_c$. Notice especially how the lowpass cutoff frequency, ω_c , becomes the *bandwidth* of the bandpass and bandstop filters.

Now suppose a lowpass transfer function, $H(s)$, is in the *cascade* form of either (6.14) for the Butterworth filter or (6.30) for the Chebyshev filter. This will turn out to be the most useful form to transform from analog to digital. Then, if we apply the substitutions (6.36) through (6.38) to one of the factors of $H(s)$, we get the terms shown in Table 6.1 for each of the other three filter types. In Table 6.1, we omit the index (m) in (6.14) and (6.30) for brevity.

In each of these translations, there is an inherent conversion of lowpass zeros and poles into highpass, bandpass, or bandstop poles and zeros. The translated zeros and poles may be derived by finding the roots of the numerators and denominators, respectively, in Table 6.1. This task is accomplished by two MATLAB functions to be discussed in Section 6.7. The function outputs are the weights of the target analog filter. We may note here, however, that the

TABLE 6.1

Frequency Translation of One Component of $H(s)$ in Cascade Form

	Butterworth	Chebyshev
Lowpass	$\frac{p}{s-p}$	$\frac{p(s-z)}{z(s-p)}$
Highpass	$\frac{s}{s-\omega_c^2/p}$	$\frac{s-\omega_c^2/z}{s-\omega_c^2/p}$
Bandpass	$\frac{ps}{s^2-ps+\omega_1\omega_2}$	$\frac{p(s^2-zs+\omega_1\omega_2)}{z(s^2-ps+\omega_1\omega_2)}$
Bandstop	$\frac{s^2+\omega_1\omega_2}{s^2-\omega_c^2s/p+\omega_1\omega_2}$	$\frac{s^2-\omega_c^2s/z+\omega_1\omega_2}{s^2-\omega_c^2s/p+\omega_1\omega_2}$

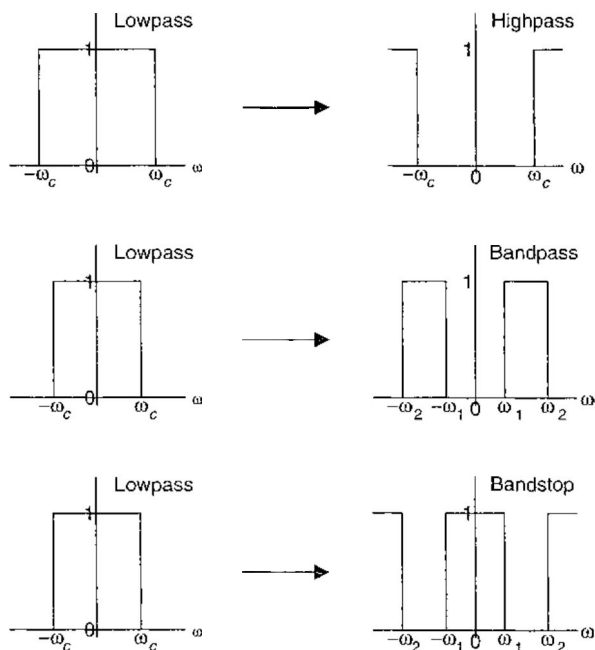
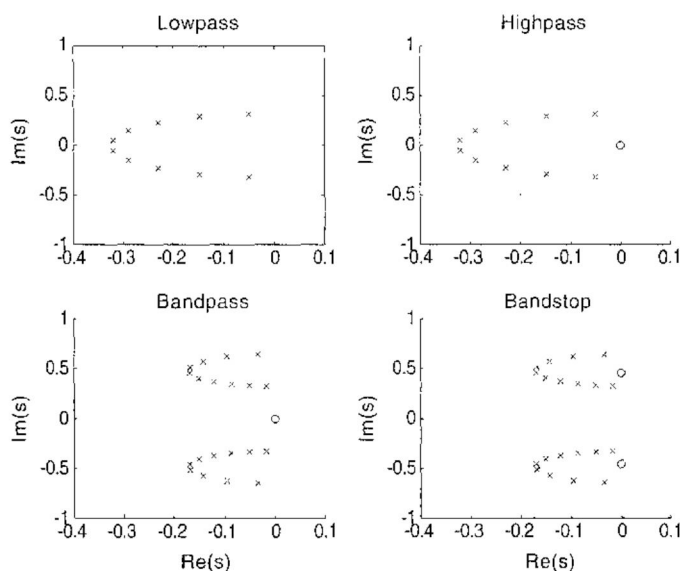


FIGURE 6.8

Illustration of the analog frequency translations in (6.36) through (6.38).

quadratic forms in the translations, where they appear in the bandpass and bandstop translations, cause the numbers of poles and zeros to double in the target filter. This and other translation features are illustrated in Figures 6.9 and 6.10, using specific examples of all six transformations in Table 6.1.

At this point, we have complete descriptions of analog Butterworth and Chebyshev filters with all four passbands. Our next step is to convert these

**FIGURE 6.9**

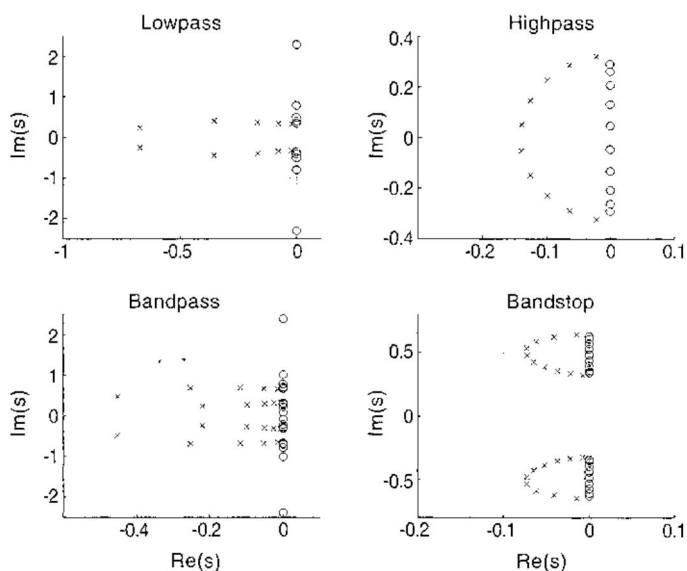
Butterworth poles on the s -plane translated from lowpass to highpass, bandpass, and bandstop configurations using (6.36) through (6.38). Parameters for the lowpass poles at upper left are $L = 10$ and $\omega_c = 0.32$. Also, $\omega_1 = 0.32$ (and hence, $\omega_2 = 0.64$) for the bandpass and bandstop translations.

to comparable digital filters. For this operation, we need one last translation. This is the *bilinear transformation* discussed in the next section. After that, we will be able to form a complete picture of IIR digital filter design.

6.6 The Bilinear Transformation

The bilinear transformation, described by Kaiser and others,^{11,12} is used to transform a continuous analog transfer function, $H(s)$, into a discrete digital transfer function, $H(z)$, and thus transform an analog filter into a digital filter. We will see that the power gain of the resulting digital filter approximates, and in one sense actually improves, the power gain of its analog counterpart, that is, any of the filters discussed in Section 6.5.

The bilinear transformation is accomplished simply by substituting a function of the discrete transform variable, z , for the continuous transform variable, s , to map points on the z -plane to points on the s -plane, and vice versa, that is, substituting a function of s for z to map from the s -plane to the z -plane.

**FIGURE 6.10**

Chebyshev poles and zeros on the s-plane translated from lowpass to highpass, bandpass, and bandstop configurations using (6.36) through (6.38). Parameters are the same as for Figure 6.9.

The mappings are as follows:

$\text{z-plane to s-plane:}$ $s \leftarrow \frac{z - 1}{z + 1}$	$\text{s-plane to z-plane:}$ $z \leftarrow \frac{1 + s}{1 - s}$	(6.39)
---	---	--------

The geometry of this transformation is easily understood if we take the squared magnitude of the right-hand expression with $s = \alpha + j\omega$:

$$|z|^2 = \frac{|1 + \alpha + j\omega|^2}{|1 - \alpha - j\omega|^2} = \frac{(1 + \alpha)^2 + \omega^2}{(1 - \alpha)^2 + \omega^2} \quad (6.40)$$

Here, we can see that if α , the real part of s , is negative, then $|z|$ is less than one, and also if α is zero, then $|z| = 1$. In other words, in the bilinear transformation from the s-plane to the z-plane,

1. The left half of the s-plane is mapped to the interior of the unit circle on the z-plane.
2. The $j\omega$ axis on the s-plane is mapped to the unit circle on the z-plane.

These are the essential properties of the bilinear transformation, which is also the simplest mapping function that produces these results.

Now suppose we convert a lowpass analog filter with transfer function $H_A(s)$ to a lowpass digital filter with transfer function $H(z)$ using the bilinear transformation with the properties just described. What happens to the filter gain in the transformation? The answer is found simply by expressing the gain function. First, from the definition (6.39), we have

$$H(z) = H_A\left(\frac{z-1}{z+1}\right) \quad (6.41)$$

From this, we can write the frequency response (gain) by substituting $z = e^{j\omega T} = e^{j2\pi v}$ into (6.41). We will use the latter form, $e^{j2\pi v}$, in order to emphasize that $H(e^{j2\pi v})$ is the *digital* gain:

$$H(e^{j2\pi v}) = H_A\left(\frac{e^{j2\pi v} - 1}{e^{j2\pi v} + 1}\right) = H_A\left(\frac{e^{j\pi v} - e^{-j\pi v}}{e^{j\pi v} + e^{-j\pi v}}\right) = H_A(j \tan(\pi v)) \quad (6.42)$$

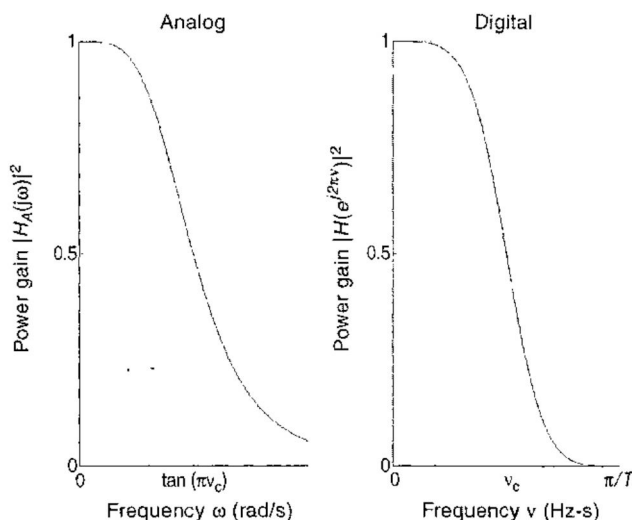
Thus, the gain of the digital filter is the same as the gain of the analog filter, but at different frequencies. We also have from (6.42) a more explicit statement of rule 2 above; that is,

Under the bilinear transformation, points given by $e^{j2\pi v}$ in the range $0 \leq v < 0.5$ on the unit circle on the z -plane map to corresponding points given by $\omega = \tan(\pi v)$ in the range $0 \leq \omega < \infty$ on the $j\omega$ -axis on the s -plane.

We recall from Chapter 4 that, with digital filters, the usual frequency range is from zero to half the sampling rate, that is, $0 \leq v \leq 0.5$. But now when the frequency goes through this range in the digital filter, we see in (6.42) that the frequency in the analog filter, that is, $\tan \pi v$, goes through the range $[0, \infty]$.

The effect of this is illustrated in Figure 6.11, in which power gain rather than dB is plotted in order to illustrate the effect clearly. In the case of a lowpass filter, the *critical frequency* is the cutoff frequency, v_c . [The power gain at cutoff for the Butterworth and type 2 Chebyshev filters is $|H(j\omega_c)|^2 = 1/2$, as shown in (6.7) and (6.19), respectively.] In Figure 6.11, we see that if the analog lowpass filter is designed to cut off at $\omega_c = \tan(\pi v_c)$ rad/s, and if this analog filter is converted to a digital lowpass filter using the bilinear transformation, then the digital filter will cut off at v_c Hz-s. Figure 6.11 simply illustrates the property of the bilinear transformation shown in (6.42), that is, the gain of the analog filter at $\tan(\pi v_c)$ rad/s is the same as the gain of the digital filter at v_c Hz-s, where v_c is any critical frequency in the range $[0, 0.5]$ Hz-s.

Another important effect of the bilinear transformation, which is also illustrated in Figure 6.11, is the mapping of analog filter gain in the range $\tan(\pi v_c) < \omega < \infty$ to digital filter gain in the range $v_c < v < 0.5$. As we can see, this part of the mapping warps the analog filter gain in a way that

**FIGURE 6.11**

Illustrating the effect of the bilinear transformation. The analog filter on the left is transformed to the digital filter on the right.

improves the digital filter gain, because the digital filter gain goes to zero at $\nu = 0.5$.

Mapping frequencies in this way also affects the rules in (6.16) and (6.34) for the necessary number of lowpass poles. Each of the critical frequencies must be mapped as just described, and so the equivalent rules for lowpass digital filters are

Necessary number of lowpass digital Butterworth poles: $L \geq \frac{(\text{maximum stopband gain in dB})}{-20 \log_{10}(\tan(\pi\nu_s)/\tan(\pi\nu_c))} \quad (6.43)$

Necessary number of lowpass digital Chebyshev poles: $L \geq \frac{6.02 - (\text{maximum stopband gain in dB})}{8.69 \cosh^{-1}(\tan(\pi\nu_s)/\tan(\pi\nu_c))} \quad (6.44)$
--

In summary, we can see that the design of an IIR lowpass digital filter with cutoff at ω_c rad/s may be accomplished in three simple steps:

1. Determine the number of poles using (6.43) or (6.44).
2. Design the analog lowpass filter, $H_A(s)$, to cut off at $\tan(\pi\nu_c)$ rad/s.
3. Transform $H_A(s)$ into $H(z)$ with the bilinear transformation, that is,

$$H(z) = H_A\left(\frac{z-1}{z+1}\right) \quad (6.45)$$

Lowpass analog filters may be transformed into highpass, bandpass, or bandstop filters as described in the preceding section. Therefore, these same rules are applicable to the design of all four types of IIR digital filters, provided all critical frequencies of the digital filter are transformed in accordance with (6.42), that is,

$$\begin{aligned}\omega_c &= \tan(\pi\nu_c) \\ \omega_1 &= \tan(\pi\nu_1) \\ \omega_2 &= \tan(\pi\nu_2)\end{aligned}\tag{6.46}$$

Thus, the rules in (6.45) may be expanded to include the design of all four types of filters:

To design an IIR digital filter:

1. Determine ω_c , ω_1 , and ω_2 , as needed using (6.46).
2. Determine the number of lowpass poles using (6.43) or (6.44).
3. Design the analog lowpass filter, $H_A(s)$, to cut off at ω_c rad/s.
4. Translate $H_A(s)$ if necessary, as shown in Table 6.1.
5. Transform $H_A(s)$ into $H(z)$ with the bilinear transformation, that is,

$$H(z) = H_A\left(\frac{z-1}{z+1}\right)\tag{6.47}$$

It is also possible, and just as easy, to exchange steps 5 and 4 and perform the frequency translations in the z -plane.¹⁶ In the next section, we describe the process (6.47) in more detail.

6.7 IIR Digital Filters

The ability described in Chapter 1, Section 1.7, to write functions that perform routine low-level tasks and to use these functions in higher-level functions, is a feature that gives MATLAB and similar high-level programming languages a great advantage over older programming languages, where defining and using functions were more difficult and restricted. We can take advantage of this capability to develop modules for the design of IIR digital filters. For example, expressions (6.43) and (6.44) for the necessary number of poles are easy enough to calculate without writing a program, but on the other hand, a MATLAB function also is easy to implement and use in other programs and has the advantage of one-time debugging. For example, the function may easily be written to make L even, so poles will be in conjugate pairs, and disallow obvious errors in

TABLE 6.2
Designing an IIR Digital Filter

Computation	MATLAB Function	Equation
# poles	$L = bw_lowpass_size(vc, vs, dB)$	(6.43)
	$L = ch_lowpass_size(vc, vs, dB)$	(6.44)
Analog weights (single-pole sections)	$[d, c] = bw_analog_weights(band, L, w1, w2)$	(6.12), (6.14)
	$[d, c] = ch_analog_weights(band, L, dB, w1, w2)$	(6.29–30), (6.36–38)
Digital weights (quadratic sections)	$[b, a] = bilin(d, c)$	(6.41)

the parameters such as specifying a critical frequency greater than half the sampling rate, etc. The function

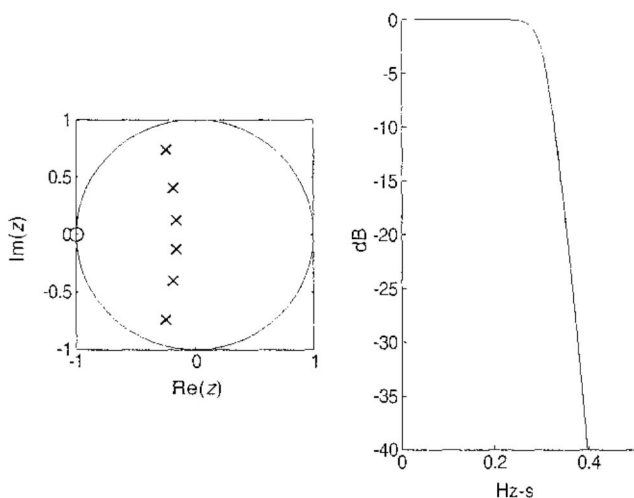
$$L = bw_lowpass_size(vc, vs, dB)$$

solves (6.43) with these constraints. All functions in this text are available with the text.

The process of designing a lowpass IIR digital filter typically begins with the selection of the filter type, which for now we assume is either Butterworth or Chebyshev type 2. Then, the process follows the course we have already described, which is summarized in Table 6.2.

There are many different ways to proceed from a lowpass analog filter to a desired digital filter, and Table 6.2 represents only one. The details of each algorithm (how it implements the corresponding equations) may be seen by examining the expressions in its complete listing. The reader may prefer to find or invent another set of algorithms. But, if the procedure in the table and the accompanying MATLAB functions are used, some remarks will be helpful. First, the frequency arguments for the first set of algorithms are in Hz-s. The frequency arguments for the second set, which are meant primarily for internal use (by other functions), are in rad, and for our purposes, are usually bilinear translations using $\tan(\pi v)$ as in (6.42). Also, for the second set of functions, the analog weights are coefficients of *single-pole* analog cascade sections and are, in general, complex. Only $L/2$ of these are generated, because the other $L/2$ are complex conjugates. It is simpler to use single-pole sections this way and pass the weights on to the final operation, which applies the bilinear translation and generates the weights of the digital filter in cascade form with quadratic sections and real coefficients. Thus, b and a are real arrays dimensioned $L/2 \times 3$, each row containing the coefficients of the corresponding quadratic filter section given in (6.1) as $b(z)/a(z)$ with $M = N = 3$.

In the procedure in Table 6.2, and others like it, with bandpass and bandstop filters, the frequency translations and bilinear transformations combine to warp the skirts of the filter gain function, so that only the gain at the critical frequencies is accurate. Thus, it is a good idea to plot the power gain and decide whether to accept the design or change one or more of the parameters.

**FIGURE 6.12**

Lowpass Butterworth digital filter designed with the procedure in (6.47). Poles and zeros are plotted on the left and power gain on the right.

A couple of examples will illustrate this. In each case, the results of the MATLAB expressions are the weights of the digital filter in cascade form with $L/2$ quadratic sections, each section having two zeros and two poles. Also, for each example, we assume that the first algorithm in Table 6.2 has provided $L = 6$ for the number of lowpass poles.

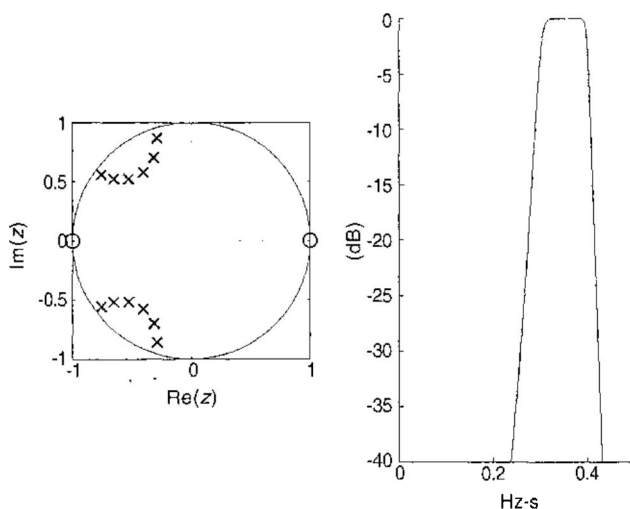
In our first example, we design a lowpass Butterworth digital filter to cut off at $\nu_c = 0.3$ Hz-s using the following expressions (see Table 6.2):

$$\begin{aligned} [d, c] &= bw_analog_weights(1, 6, \tan(\pi*0.3)); \\ [b, a] &= bilin(d, c); \end{aligned} \quad (6.48)$$

The results are shown in Figure 6.12 in terms of a pole-zero plot and a power gain plot. Note that the bilinear transformation creates a zero of order L at $z = -1$, even though the lowpass analog filter has only poles. Notice also how the distribution of poles causes the gain to be uniformly high until the frequency ν approaches 0.3 Hz-s, that is, $e^{j2\pi\nu}$ approaches $e^{j0.6\pi}$, where the polar angle is 108° . Above this frequency, the operating point moves toward the six zeros at $z = -1$, and the gain decreases rapidly toward zero.

In the second example, with results shown in Figure 6.13, we produce the weights of a digital Butterworth bandpass filter with passband from $\nu_1 = 0.3$ to $\nu_2 = 0.4$ using the following expressions:

$$\begin{aligned} [d, c] &= bw_analog_weights(3, 6, \tan(\pi*0.3), \tan(\pi*0.4)); \\ [b, a] &= bilin(d, c); \end{aligned} \quad (6.49)$$

**FIGURE 6.13**

Bandpass Butterworth digital filter designed with the procedures in (6.47). Poles and zeros are plotted on the left and power gain on the right.

On account of the quadratic substitution for s in (6.37), which causes L to increase from 6 to 12, the bandpass filter has six quadratic sections—twice as many as the lowpass filter. On the pole-zero plot in Figure 6.13, notice how the poles cover the passband from $\Omega_1 = 0.6\pi$ to $\Omega_2 = 0.8\pi$ rad, and the -3 dB points on the power gain curve are at $v_1 = 0.3$ and $v_2 = 0.4$. In this case, there are also 12 zeros, six at $z = 1$ and six at $z = -1$.

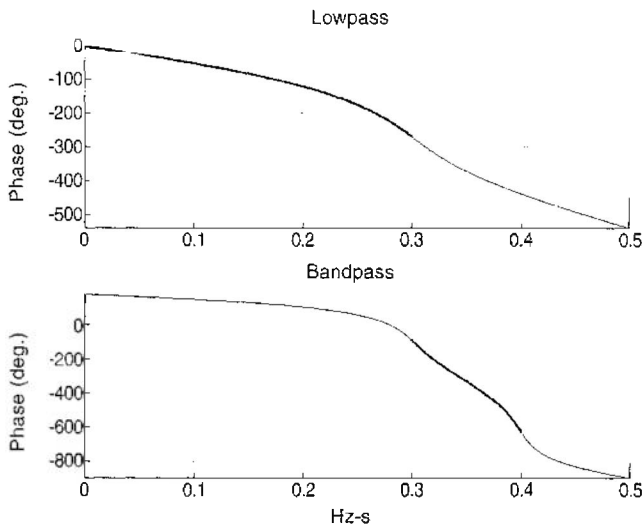
Phase characteristics for these two examples are illustrated in Figure 6.14, with the passband phases signified by heavy lines. The passband phases are seen to be nearly linear and would be acceptable in many filtering applications. But if absolutely linear phase is required, we must revert either to an FIR filter or the time-reversal technique described in Section 6.2.

Instead of the two expressions in (6.48) and (6.49), one may wish to write a single function that computes analog weights and transforms these to digital weights using the bilinear transformation. Two such functions are provided along with the other functions in Table 6.2. These are

$$\begin{aligned} [b, a] &= bw_weights(band, L, v1, v2) \\ [b, a] &= ch_weights(band, L, dB, v1, v2) \end{aligned} \quad (6.50)$$

If you examine the listings of either of these functions, you will see that most of the effort in composing the function is in the comments. But if your memory is not perfect or if the function is for use by others as well as yourself, it is justified.

Techniques for IIR filtering, that is, using the IIR digital filters in this chapter to filter sample vectors, have been described in Chapter 4. Any of

**FIGURE 6.14**

Phase characteristics of the lowpass and bandpass filters used for Figures 6.12 and 6.13. Phase in the passband of each filter is shown as a heavier line.

the algorithms discussed in Chapter 4, including the MATLAB *filter* function, may be applied using the filters in this chapter. The MATLAB *filter* function is the best choice if MATLAB is being used; however, the current version of *filter* requires the direct form of $H(z)$ rather than the cascade form preferred in this chapter. To implement the cascade form, the function

$$y = \text{filters}(b, a, x)$$

may be used. This function allows b and a to be arrays, with each row containing the weights of one section of the cascade structure. In other words, *filters* accept weights as computed by the functions in (6.50). Internally, *filters* simply uses *filter* to implement each section of the cascade structure.

The exercises contain additional examples of designing and applying IIR filters and filtering using the techniques described in this section.

6.8 Other Types of IIR Filters

Besides the four standard types of IIR filters, there is an unlimited variety of different transfer functions that one may wish to invent for special purposes. We will give only two examples here, but in general, a useful way to think about obtaining a given amplitude gain or phase characteristic is to

imagine how poles and zeros might be placed on the z -plane to achieve the desired results. The first example illustrates this process.

Suppose we need a *digital resonator*, that is, a digital filter that has high gain at a specified frequency and low gain at all other frequencies. We saw in Chapter 4, Section 4.5, how gain is affected by distances from the operating point, $e^{j\omega T}$ on the unit circle, to poles and zeros on the z -plane. That is, the gain is high when the operating point is close to a pole and low when the operating point is near a zero. Now suppose we want a resonator that resonates at $\omega = \omega_0$. Then, it is reasonable to place a pole just inside the unit circle (for stability), say at radius $r = 1 - \epsilon$. This pole, along with its conjugate, would give us the transfer function

$$H_r(z) = \frac{z^2}{(z - re^{j\omega_0 T})(z - re^{-j\omega_0 T})} = \frac{z^2}{z^2 - 2rz \cos \omega_0 T + r^2} \quad (6.51)$$

Now suppose we wish to place zeros all around on the unit circle to suppress the gain at frequencies away from $\omega_0 T$. We could do this with

$$H_z(z) = 1 - z^{-N} \quad (6.52)$$

which has zeros at $(1)^{1/N}$; that is, in a manner similar to (6.9),

$$\text{zeros at } z = e^{j2\pi n/N}, \quad n = 0, 1, \dots, N-1 \quad (6.53)$$

We then construct the digital resonator as the cascade combination $H_z(z)H_r(z)$ shown in Figure 6.15, with a constant gain A to make the gain equal to one at resonance, that is,

$$A = \frac{1}{|H_z(e^{j\omega_0 T})H_r(e^{j\omega_0 T})|} \quad (6.54)$$

Thus, the digital resonator is a simple structure consisting of the delay section (6.52) followed by the two-pole IIR filter section (6.51). An example with $N = 16$, $r = 0.99$, and $\omega_0 T = 2.5(2\pi/N)$, is given in Figure 6.16, which shows pole-zero and power gain plots for the resonator. Note that the pole is placed halfway between two zeros and just inside the unit circle at $r = 0.99$, so the gain is nearly uniform on either side of ω_0 .

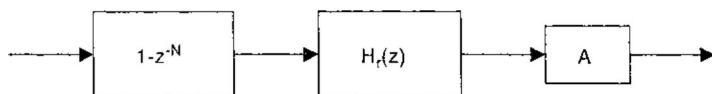


FIGURE 6.15

Digital resonator: cascade combination of $H_z(z)$ in (6.52), $H_r(z)$ in (6.51), and constant gain A in (6.54).

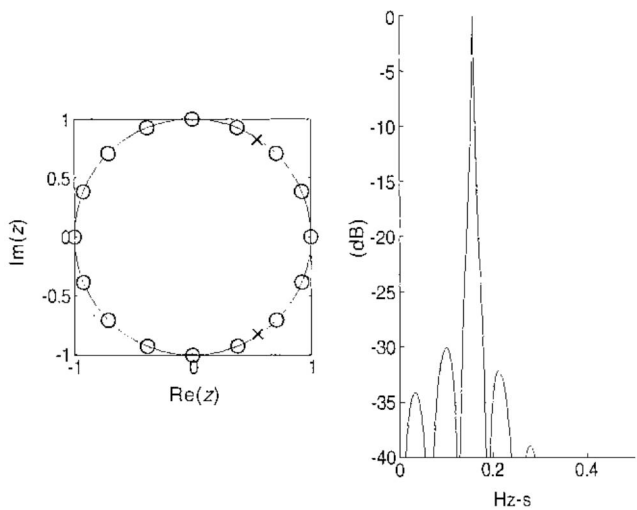


FIGURE 6.16
Pole-zero and power gain plots for the configuration in Figure 6.15 with $N = 16$ zeros, pole radius $r = 0.99$, and resonant frequency $2.5(2\pi/N)$ rad.

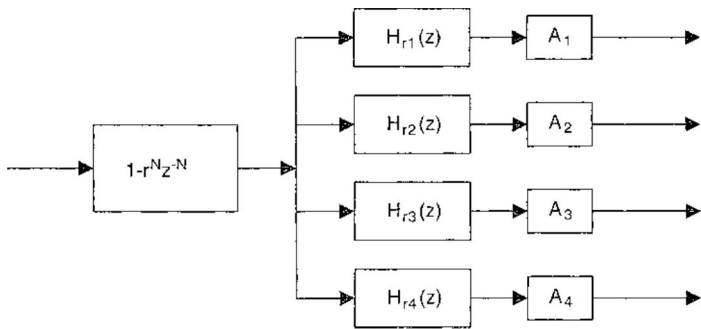
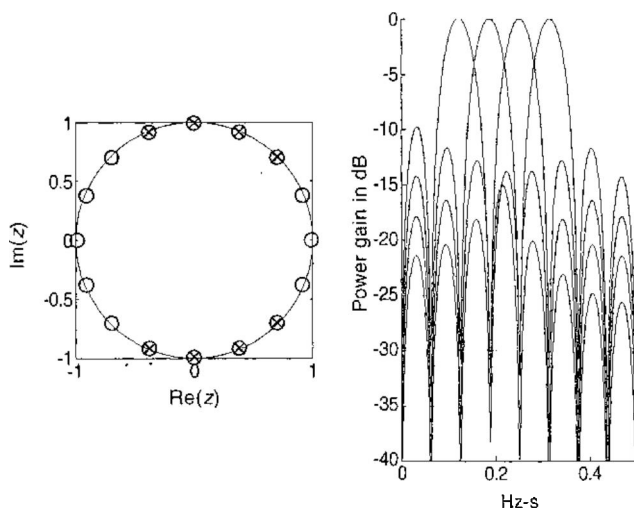


FIGURE 6.17
Comb filter configuration described by (6.51) through (6.54).

The *comb filter* shown in Figure 6.17 is another type of IIR resonator, in which several units in the form of $H_i(z)$ in (6.51) and (6.54) are connected to a single FIR delay:

$$H_z(z) = 1 - r^N z^{-N} \tag{6.55}$$

The factor r^N causes the zeros to move in from the unit circle to a circle of radius r . Each pole of the comb filter is placed directly on a zero, and

**FIGURE 6.18**

Pole-zero and power gain plots for the configuration in Figure 6.17 with $N = 16$ zeros, pole and zero radii $r = 0.99$, and resonant frequencies at $[2:5] * (2 * \pi/N)$ rad.

therefore, cancels its effect on gain, thus producing a resonant peak at the corresponding frequency. The result is shown in Figure 6.18—a “comb” filter with outputs in adjacent frequency bands.

The *spectrogram* is an interesting application of the comb filter or any similar set of bandpass filters. It is useful with *nonstationary* signals (signals with spectral properties that change with time), which are discussed in the next chapter. An example of a nonstationary signal, in this case, a “chirping” signal that increases in both amplitude and frequency, is shown in Figure 6.19. A spectrogram of the signal, consisting of the outputs of a comb filter with eight bands, is shown in Figure 6.20. With a properly constructed spectrogram like this, we can observe the arrival of different frequency components of a signal at different times, which may be very useful with certain types of data, such as acoustic or seismic waves.

Our second example of a different type of IIR filter is the *all-pass filter*, which has constant gain at all frequencies. Suppose we place poles at p and p' (the conjugate of p) on the z -plane, and then, to compensate for the contributions of these to the amplitude gain, we place zeros at $1/p$ and $1/p'$ and construct the following filter:

$$H(z) = \frac{|p|^2(z - 1/p)(z - 1/p')}{(z - p)(z - p')} = \frac{(pz - 1)(p'z - 1)}{(z - p)(z - p')} \quad (6.56)$$